



UNIVERSITY OF CAGLIARI

Department of Engineering

MASTER'S DEGREE IN "COMPUTER ENGINEERING,
CYBERSECURITY AND ARTIFICIAL INTELLIGENCE"

**Advanced Embedded System
Lab 5 - DNN on MNIST**

Advisor:
Prof. Paolo Meloni

Authors:
Lello Molinaro

Accademic Year 2025/2026
Cagliari

January 19, 2026

Contents

1	Introduction	3
2	Target Platform and Development Environment	5
3	DNN Architecture Analysis	7
3.1	Laboratory Assignment Context	7
3.2	Baseline Network	7
3.3	Custom Network Topology (Group 4)	10
4	Fixed-Point DNN Implementation	12
4.1	Data Representation and Q-Format	13
4.2	Multiply-Accumulate Operations	13
4.3	Saturation and Overflow Control	13
4.4	Activation Functions	14
4.5	Compile-Time Configurability	14
4.6	Benefits of the Fixed-Point Design	14
5	UART-Based DNN Microserver	16
5.1	Input Streaming Architecture	16
5.2	Blocking Reception and Deterministic Behavior	18
5.3	Functional Validation Using Embedded Test Images	18
6	Execution Time Measurement	20
6.1	Measurement Methodology	20
6.2	Warm-Up and Steady-State Measurements	21
6.3	Separation of Communication and Computation	21
6.4	Consistency with Previous Laboratories	22
6.5	Per-Layer Profiling	22
6.6	Communication Versus Computation	24
7	Optimization Tips and Custom Enhancements	25
7.1	Input Data Optimization (Tip 1)	25
7.2	Weight and Bias Representation Optimization (Tip 2)	26
7.3	Compile-Time Configurability and Experimental Fairness	27
7.4	Scalability and Impact on System Performance	28
8	Validation and Debugging of Input Data Consistency	29
8.1	Hypothesis and Debugging Strategy	32
8.2	Offline Pixel-Wise Consistency Verification	32
8.3	Analysis of the Digit “5” Anomaly	32
8.4	Cross-Validation Using a Python Golden Model	33
8.5	Implications for Experimental Correctness	34
9	Performance Evaluation	35

9.1	Experimental Setup and UART-Based Input Methodology	36
9.2	Runtime Performance Results	36
9.3	Compute-Only Throughput	37
9.4	Real-Time Capability Analysis	38
9.5	Baseline Performance Results	39
9.6	Performance Results with Optimization Tips Enabled	39
9.6.1	Compute Performance	39
9.6.2	Impact on Communication Latency	40
9.6.3	Numerical Robustness and Stability	40
9.6.4	Classification Behavior under Fixed-Point Optimizations	40
9.6.5	Discussion on Optimized Configuration	41
9.7	Overall Discussion	42
10	Conclusions	44
11	Appendix – Preprocessing Tools and Optimization Scripts	46
11.1	Overview of the Optimization Workflow	46
11.2	Input Image Quantization (TIP 1)	47
11.2.1	Python Script for Q8.8 to Q0.8 Conversion	47
11.3	Weight and Bias Conversion (TIP 2)	48
11.3.1	Python Script for Weight and Bias Conversion	48
11.4	Baseline vs Custom Network Validation	50
11.4.1	Offline Image Consistency Verification	50
11.4.2	Python DNN Golden Model	51
11.5	Runtime Output Log of Experimental Results	54

Chapter 1

Introduction

The increasing adoption of Artificial Intelligence techniques in embedded and resource-constrained systems has highlighted the importance of efficient software–hardware co-design methodologies. In particular, the deployment of Deep Neural Networks (DNNs) on embedded platforms poses significant challenges in terms of computational complexity, memory usage, data movement, and real-time constraints.

This laboratory activity (Lab 5) builds upon the concepts and methodologies introduced in Laboratory 3 and Laboratory 4, extending low-level performance measurement and software optimization techniques to the implementation of a complete DNN inference pipeline. While previous laboratories focused on isolated computational kernels (e.g., FIR filtering and parallel vector operations), this work addresses the integration of computation, communication, and performance profiling within a single embedded application.

The report describes the implementation of a DNN-based microserver for handwritten digit recognition (MNIST) running on the Zybo Z7 platform. The application performs real-time inference on images streamed from a host PC via UART and is specifically designed according to embedded system constraints, without relying on hardware accelerators or dedicated neural processing units.

The target platform is based on the Zynq-7000 System-on-Chip, where an ARM Cortex-A9 Processing System executes a fully software-based inference pipeline. The adopted approach emphasizes explicit control over data representation, memory organization, and execution flow, which are key aspects in the context of Advanced Embedded Systems.

From an architectural perspective, the proposed implementation explicitly follows a reduced instruction set execution model, where computation is expressed through regular load–compute–store patterns and deterministic control flow. Although the target processor is an ARM Cortex-A9, the adopted execution model is intentionally structured according to a *RISC-style programming paradigm*, as introduced in the Advanced Embedded Systems course. In particular, the implementation emphasizes explicit load–compute–store sequences, simple arithmetic instructions, regular loop structures, and predictable control flow. References to MIPS are therefore conceptual and pedagogical, rather than architectural, and are used to highlight instruction-level regularity and timing predictability rather than ISA-specific features.

The implemented DNN processes grayscale images of size 28×28 pixels and classifies each input sample as a digit in the range from 0 to 9. Inference is carried out using fixed-point arithmetic, allowing the elimination of floating-point operations and ensuring deterministic execution behavior. Multiple fixed-point configurations are supported and selected at compile time, including a baseline reference configuration and optimized modes targeting communication efficiency and numerical robustness.

In addition to functional correctness, particular attention is devoted to the experimental evaluation of performance. Execution times are measured using the ARM Cortex-A9 Global Timer

through the `XTime_GetTime()` API, enabling fine-grained profiling of individual stages such as UART-based data reception, fully connected layers, activation functions, and result processing. This measurement-driven approach allows a quantitative assessment of real-time capability and highlights the impact of system-level optimizations.

Finally, the report presents an extended implementation based on a custom DNN topology assigned to Group 4, demonstrating the scalability and modularity of the proposed software framework. The combination of architectural analysis, fixed-point design, and experimental profiling provides a comprehensive view of how DNN inference can be effectively mapped onto an embedded ARM-based platform under realistic I/O and performance constraints.

Chapter 2

Target Platform and Development Environment

The target platform for this laboratory activity is the Processing System (PS) of the Zynq-7000 System-on-Chip, specifically the ARM Cortex-A9 core available on the Zybo Z7 development board. The Zynq-7000 architecture integrates a dual-core ARM processing system with programmable logic on the same device, enabling a wide range of hardware–software co-design solutions.

The entire Deep Neural Network inference pipeline is executed exclusively on the ARM Cortex-A9 processor, without exploiting the programmable logic or any dedicated hardware accelerator. This design choice is deliberate and aligns with the objectives of the laboratory, which focus on evaluating software efficiency, fixed-point arithmetic, and performance profiling under embedded constraints. By avoiding hardware acceleration, the implementation allows direct analysis of instruction-level execution, memory access patterns, and real-time behavior on a general-purpose embedded processor.

The software is developed as a standalone application using the Xilinx Software Development Kit (SDK). The standalone execution model avoids the overhead introduced by an operating system, providing direct access to hardware peripherals and enabling precise and deterministic timing measurements. This execution environment is particularly suitable for low-level optimization and performance evaluation tasks, which are central topics in the Advanced Embedded Systems course.



Figure 2.1: Xilinx Software Development Kit (SDK) environment used for embedded application development.

Communication between the host computer and the embedded platform is implemented through a UART interface. The UART channel is used to stream MNIST input images from the host PC to the board and to transmit classification results back to the host. This communication model reflects a realistic embedded scenario, where external data sources are often connected through low-bandwidth serial interfaces. The use of UART also enables clear separation between communication latency and computation time, which is essential for meaningful performance analysis.

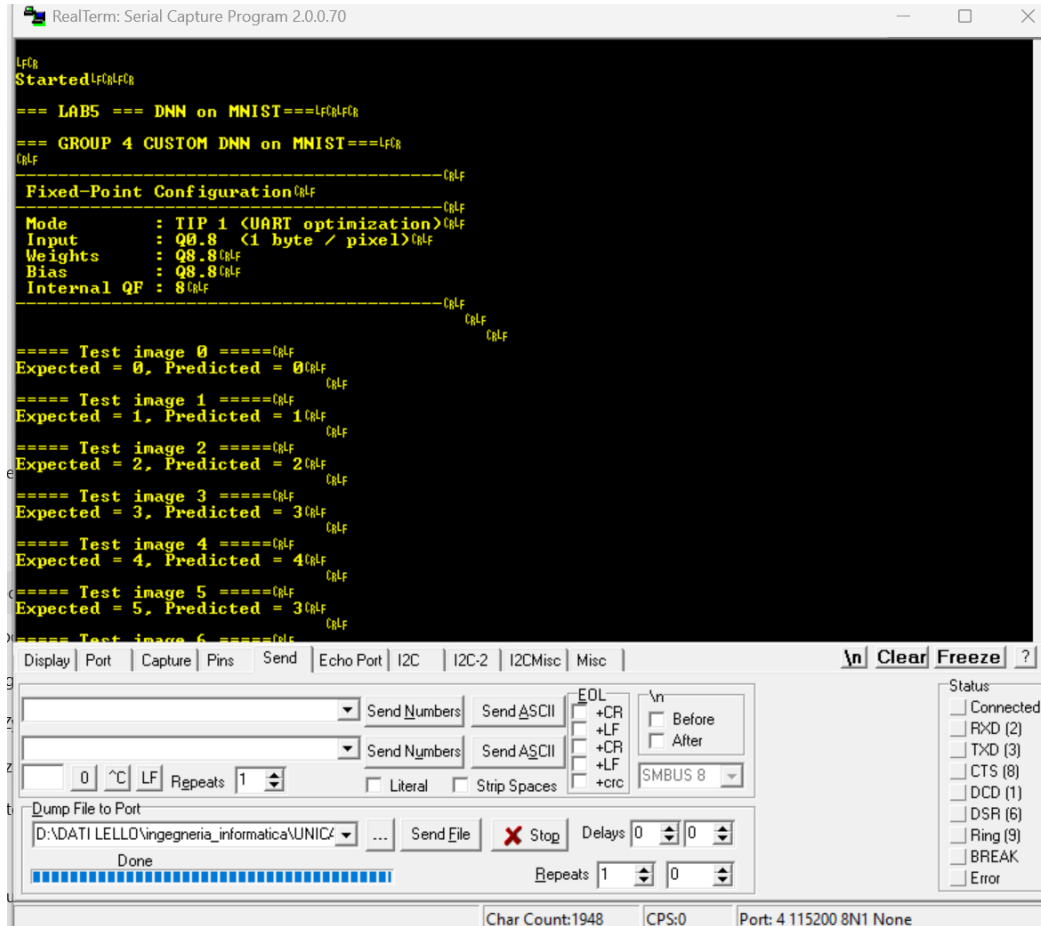


Figure 2.2: RealTerm serial/TCP terminal used to transmit binary MNIST data streams over UART.

All timing measurements are obtained using the 64-bit ARM Global Timer available in the Processing System. Execution times are measured through the `XTime_GetTime()` API, which provides a hardware-based, free-running timing source independent of software overhead. This approach enables microsecond-level profiling of individual stages of the application, including UART data reception, DNN inference, and result processing.

Chapter 3

DNN Architecture Analysis

3.1 Laboratory Assignment Context

Laboratory 5 addresses the implementation and evaluation of a DNN-based microserver on the Zybo Z7 platform for handwritten digit recognition using the MNIST dataset.¹ The assignment requires the analysis of a provided DNN model in ONNX format, the manual composition of the inference pipeline in C, and the integration of UART-based image streaming within a cyclic execution loop. Each iteration performs image reception, DNN inference, and result transmission, while execution time measurements are collected to quantify communication and computation overheads.

3.2 Baseline Network

The baseline DNN topology was analyzed by inspecting the corresponding ONNX model using the Netron visualization tool².

The network is composed exclusively of fully connected layers, preceded by a flattening stage that converts the two-dimensional input image into a one-dimensional feature vector.

The input to the network is a grayscale MNIST image of size 28×28 pixels, which is flattened into a vector of 784 elements, as shown in Figure 3.1.

¹<https://sites.google.com/view/lab-instructions-aes/home-page/lab5-dnn-on-mnist>

²<https://netron.app>

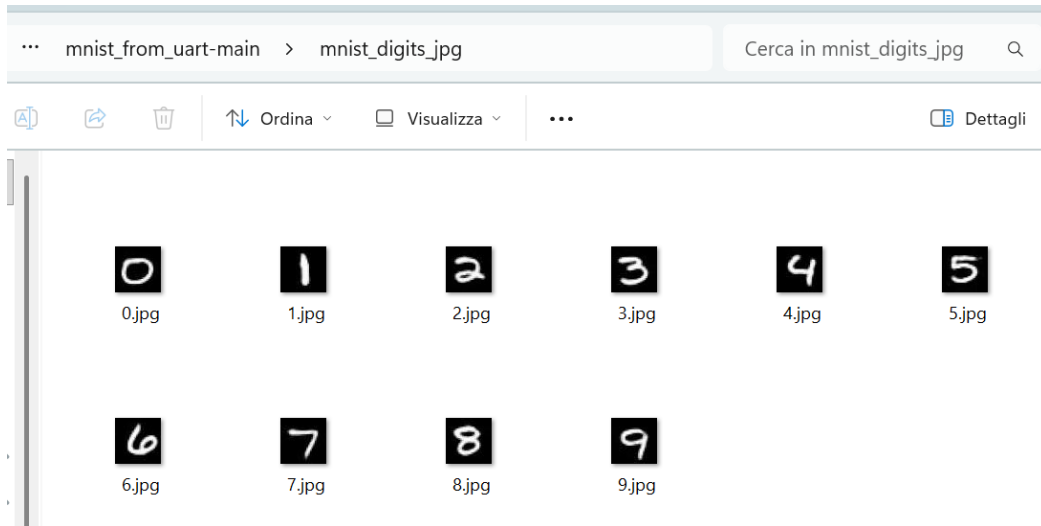


Figure 3.1: Example of a MNIST handwritten digit image used as input for the DNN inference.

The flattened input vector is processed by a sequence of fully connected layers with decreasing dimensionality, followed by non-linear activation functions. Specifically, the baseline network consists of the following stages:

- a fully connected layer mapping 784 input features to 64 neurons,
- a fully connected layer mapping 64 features to 32 neurons,
- a final fully connected layer mapping 32 features to 10 output classes.

Each intermediate fully connected layer is followed by a ReLU activation function, while the output layer produces a vector of 10 scores corresponding to the digit classes from 0 to 9. This output vector is subsequently processed by a simple classification stage that selects the class associated with the maximum score.

The baseline architecture represents a compact and regular reference model for MNIST classification. Its fully connected structure results in predictable memory access patterns and a deterministic execution flow, making it well suited for implementation on embedded processors without specialized hardware acceleration.

Figure 3.2 shows the baseline DNN topology as visualized using Netron.

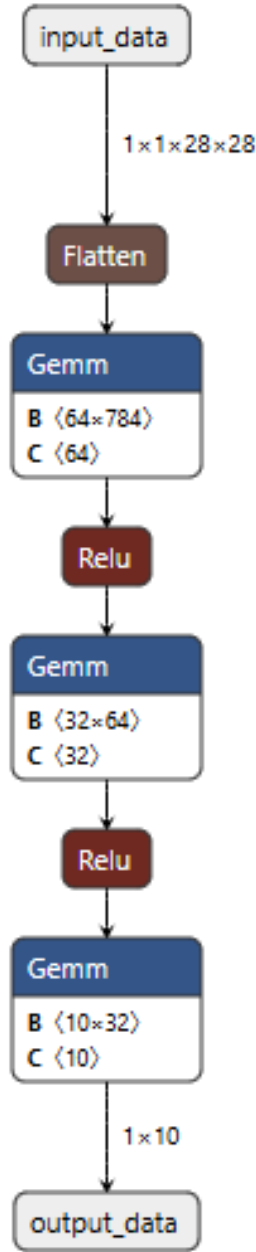


Figure 3.2: Baseline MNIST DNN topology visualized using Netron.

The simplicity of the baseline network allows a direct one-to-one mapping between the graphical representation provided by Netron and the corresponding C implementation (reachable at Github repository)³. Each fully connected layer is explicitly implemented as a pair of nested loops performing multiply-accumulate operations, while activation functions are implemented as separate processing stages. This clear separation of stages facilitates debugging, profiling, and further optimization.

For a fully connected layer with N input features and M output neurons, the computational workload can be expressed in terms of multiply-accumulate operations as:

³<https://github.com/lmolinario/AES>

$$\text{MACs} = N \times M$$

In the baseline network, the first fully connected layer ($784 \rightarrow 64$) therefore requires:

$$784 \times 64 = 50,176 \text{ MAC operations}$$

which dominates the overall computational cost of the inference pipeline.

3.3 Custom Network Topology (Group 4)

In addition to the baseline network, a custom DNN topology was implemented according to the group-specific assignment (Group 4). The custom network extends the baseline architecture by introducing an additional hidden fully connected layer, thereby increasing the depth of the network and the overall computational workload.

The custom topology is composed of the following layers:

- a fully connected layer mapping 784 input features to 64 neurons,
- a fully connected layer mapping 64 features to 32 neurons,
- a fully connected layer mapping 32 features to 16 neurons,
- a final fully connected layer mapping 16 features to 10 output classes.

As in the baseline case, ReLU activation functions are applied after each intermediate layer, while the output layer produces a vector of class scores. The introduction of an additional hidden layer increases both the number of trainable parameters and the number of arithmetic operations required for a single inference.

Figure 3.3 illustrates the custom DNN topology as visualized using Netron.

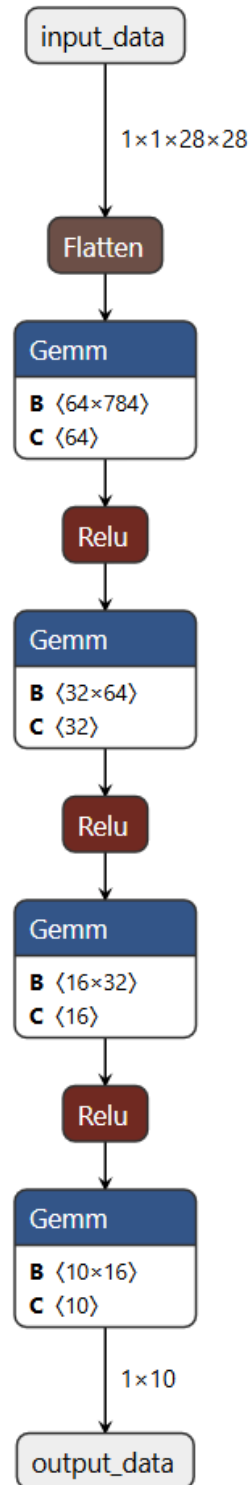


Figure 3.3: Custom DNN topology for Group 4 visualized using Netron.

Chapter 4

Fixed-Point DNN Implementation

The Deep Neural Network inference is implemented entirely using fixed-point arithmetic. All weights, biases, inputs, and intermediate activations are represented using 16-bit signed integers (`int16`), enabling the elimination of floating-point operations on the ARM Cortex-A9 processor.

This design choice is particularly suitable for embedded systems, where floating-point units may introduce additional latency, non-deterministic execution times, and higher energy consumption. By relying exclusively on integer arithmetic, the implemented inference pipeline achieves deterministic execution behavior and explicit control over numerical precision, scaling, and overflow.

Figure 4.1 illustrates the fixed-point accumulation strategy adopted in the fully connected layers.

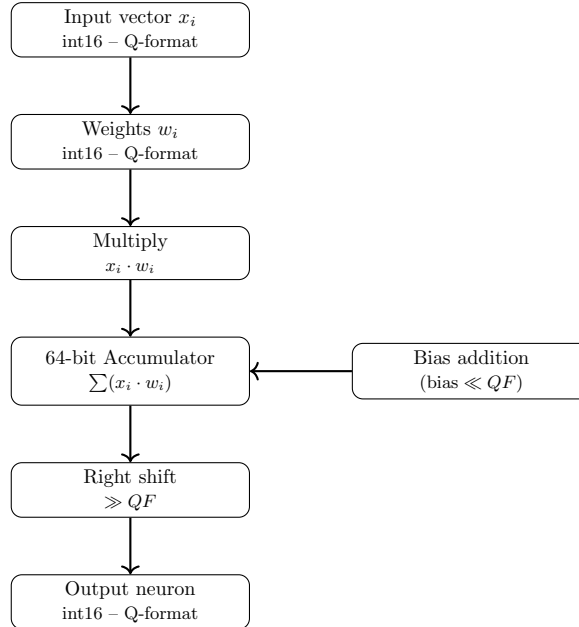


Figure 4.1: Conceptual fixed-point multiply-accumulate (MAC) operation used in the fully connected layers. Inputs and weights are multiplied using fixed-point arithmetic, accumulated with extended 64-bit precision, and scaled back to a 16-bit output representation through an explicit right-shift operation.

4.1 Data Representation and Q-Format

All real-valued quantities in the DNN are encoded using a Q-format fixed-point representation. In the baseline configuration, network parameters and activations are represented using a Q8.8 format, where 8 bits are allocated to the integer part and 8 bits to the fractional part.

The Q8.8 format provides a good trade-off between numerical precision and dynamic range for MNIST classification, while remaining fully compatible with 16-bit integer arithmetic. This representation is used as the reference configuration for functional validation and performance comparison.

In addition to the baseline format, alternative fixed-point configurations are supported through compile-time selection. In particular, the optimized configuration (Tip 2) adopts a Q1.7 format for weights and biases, reducing the fractional precision by one bit with respect to Q8.8 and applying offline saturation to a bounded domain. This represents a controlled quantization strategy aimed at improving numerical robustness and dynamic-range control while preserving sign symmetry and full compatibility with 16-bit arithmetic.

The selected Q-format is controlled by the compile-time constant $\mathbf{QF}$, ensuring deterministic behavior and reproducible measurements across different runs.

Operationally, the Q-format is implemented by scaling real-valued quantities by a factor of $2^{\mathbf{QF}}$ prior to quantization, and by applying a symmetric right shift after accumulation to restore the target fixed-point representation. The offline conversion process used to generate Q1.7 parameters from the original Q8.8 representation is described in Appendix 11.3.

4.2 Multiply–Accumulate Operations

Each fully connected layer is implemented explicitly as a sequence of multiply–accumulate (MAC) operations. For each output neuron, the dot product between the input vector and the corresponding weight vector is computed.

To preserve numerical accuracy and avoid intermediate overflow, accumulation is performed using a 64-bit signed integer accumulator. Bias terms are added at the beginning of the accumulation process and are aligned according to the selected Q-format, ensuring that bias contribution is accumulated with the same numerical precision as the weighted sum. After completion of the accumulation, the result is scaled back to the target fixed-point representation by means of a right-shift operation proportional to the fractional length. The right-shift operation corresponds to a division by $2^{\mathbf{QF}}$ and restores the fixed-point scaling expected by subsequent layers.

This implementation directly reflects the arithmetic structure of the DNN and allows fine-grained control over scaling and precision at each layer. The adopted fixed-point MAC strategy is conceptually illustrated in Figure 4.1.

At instruction level, each MAC operation maps naturally to a RISC-style instruction sequence, composed of operand loads, an integer multiplication, and an integer addition. This load–compute–store execution pattern is fully consistent with the MIPS architectural model, where arithmetic instructions operate exclusively on register operands and memory accesses are explicitly managed through load and store instructions.

4.3 Saturation and Overflow Control

To ensure predictable behavior in an embedded execution context, the implementation explicitly considers saturation and overflow control. Potential overflow conditions may arise during the accumulation phase of the multiply–accumulate operations, where partial sums grow proportionally to the number of input features and weight magnitudes. A saturation function is provided to clamp accumulated values exceeding the representable range of a 16-bit signed integer.

In the tested configurations (Q8.8 and Q1.7), no overflow conditions were observed during experimental validation across all evaluated inputs and network layers. For this reason, saturation is not actively applied in the main inference loop in order to avoid unnecessary overhead and conditional branches that could negatively impact execution time predictability. However, the saturation mechanism is intentionally retained in the codebase to support future extensions, such as reduced-precision implementations or more aggressive scaling strategies.

This design choice balances robustness and performance, preserving deterministic execution while maintaining extensibility for future architectural or numerical modifications. It is worth noting that the saturation to the signed 8-bit range discussed in the context of Tip 2 applies exclusively to offline preprocessing of network parameters, while all runtime computations operate on 16-bit fixed-point data with 64-bit accumulation.

4.4 Activation Functions

Non-linear activation functions are implemented explicitly using integer arithmetic. In particular, the Rectified Linear Unit (ReLU) function is applied after each intermediate fully connected layer. The ReLU function is particularly well suited for fixed-point implementations, as it does not require multiplications, divisions, or rescaling operations.

The ReLU operation is implemented as a simple comparison followed by a conditional assignment that sets negative values to zero. This implementation is computationally inexpensive and introduces negligible overhead compared to the fully connected layers, where the dominant multiply-accumulate workload is concentrated. As a result, the execution time of the activation stage is data-independent and fully deterministic, which is a desirable property in real-time embedded systems.

4.5 Compile-Time Configurability

All fixed-point configurations are selected at compile time using preprocessor macros. Configuration selection is implemented through preprocessor directives that statically include the appropriate data representations and arithmetic paths at compilation time. This approach avoids runtime branching, ensures deterministic execution paths, and enables fair performance comparison between different configurations.

Specifically, the implementation supports:

- a baseline configuration with Q8.8 input data and parameters,
- an optimized configuration with compact Q0.8 input data (Tip 1),
- an extended optimized configuration with Q1.7 weights and biases (Tip 1 + Tip 2).

The compile-time selection mechanism guarantees that each configuration is evaluated under identical execution conditions, excluding configuration-related control flow from runtime execution and isolating the impact of data representation choices on performance and robustness.

4.6 Benefits of the Fixed-Point Design

The fixed-point implementation provides several advantages in the context of embedded systems:

- complete elimination of floating-point operations and reliance on the FPU,
- reduced instruction count and computational overhead,
- deterministic execution times suitable for cycle-accurate profiling using the ARM Global Timer,
- explicit control over numerical precision, scaling, and overflow behavior,

-
- compatibility with communication-oriented optimizations and system-level profiling.

All these advantages are directly reflected in the experimental results presented in the following chapters, where deterministic timing behavior and stable cycle-level measurements are consistently observed. These properties are fundamental for achieving reliable, efficient, and predictable DNN inference on a general-purpose embedded processor, demonstrating that fixed-point software-based implementations can meet real-time constraints even in the absence of dedicated hardware accelerators.

Chapter 5

UART-Based DNN Microserver

This chapter describes the communication architecture adopted to implement the DNN microserver. The microserver is responsible for receiving input data from an external host, executing the neural network inference, and returning the classification result to the host system.

An example overview of the adopted UART-based microserver architecture is shown in Figure 5.1. The system follows a request–response microserver model, where the host explicitly triggers inference by transmitting input samples, and the embedded platform responds with the corresponding classification output.

Communication between the host PC and the embedded platform is implemented using a UART-based interface. This design choice reflects a realistic embedded scenario, where data are often acquired through low-bandwidth serial links, and enables a direct comparison with the communication models adopted in previous laboratory assignments, while maintaining full control over data transfer timing and synchronization.

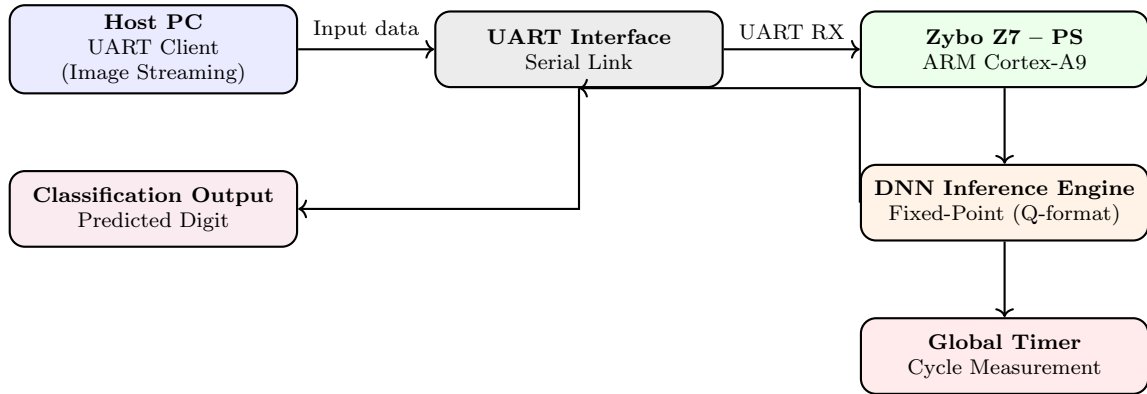


Figure 5.1: UART-based DNN microserver architecture. The host PC streams input data to the embedded platform, which performs fixed-point inference and returns the classification result over the same serial interface.

5.1 Input Streaming Architecture

MNIST images are streamed from a host PC to the Zybo Z7 board via UART as raw pixel values. Each image consists of 784 samples corresponding to a grayscale image of size 28×28 pixels.

Data reception is implemented using a blocking strategy, where the application waits until the complete image has been received before starting the inference process. At implementation

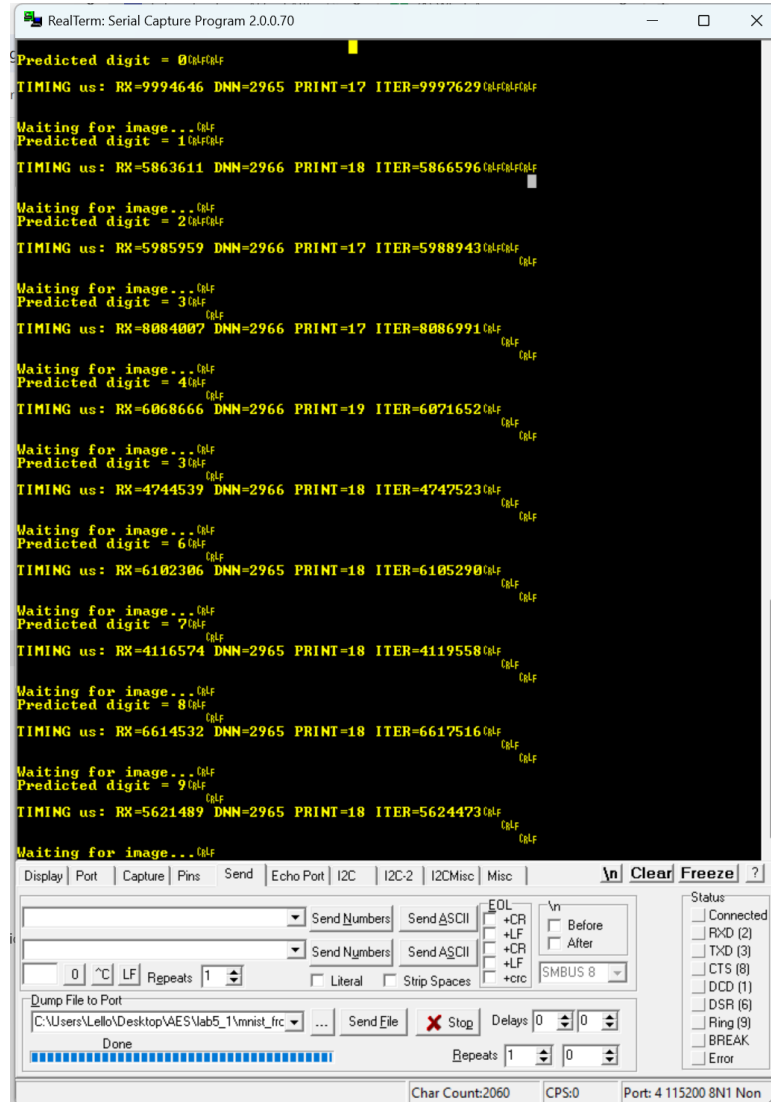
level, this behavior is realized through a blocking UART receive loop that explicitly waits for all 784 pixel values before enabling the inference stage.

This approach simplifies synchronization between communication and computation and ensures a clear separation between input acquisition and neural network execution.

Each pixel is transmitted as a fixed-point value and reconstructed on the embedded platform according to the active fixed-point configuration. This design allows seamless integration with the fixed-point inference pipeline described in the previous chapter and avoids any implicit data conversion during runtime.

In the baseline configuration, each pixel is transmitted using two bytes, corresponding to a Q8.8 fixed-point representation. While straightforward, this approach introduces significant communication latency due to the limited UART bandwidth.

Figure 5.2 shows an example of the UART-based streaming process for a single MNIST image, including raw data reception, inference execution, and classification result transmission during the validation phase.



The screenshot displays the RealTerm Serial Capture Program interface. The main window shows a log of serial data with alternating yellow and black lines. The yellow lines represent the model's output, showing predicted digits (0 through 9) and timing information (RX, DNN, PRINT, ITER). The black lines represent the raw data received from the device, which includes the text "Waiting for image...". The interface also features a control panel at the bottom with tabs for Display, Port, Capture, Pins, Send, Echo Port, I2C, I2C-2, I2CMisc, and Misc. The Status panel on the right shows the connection status and various hardware signals. The bottom status bar indicates a character count of 2060, a baud rate of 4115200, and a port of 4115200 8N1 Non.

```
Predicted digit = 0
TIMING us: RX=9994646 DNN=2965 PRINT=17 ITER=9997629
Waiting for image...
Predicted digit = 1
TIMING us: RX=5863611 DNN=2966 PRINT=18 ITER=5866596
Waiting for image...
Predicted digit = 2
TIMING us: RX=5985959 DNN=2966 PRINT=17 ITER=5988943
Waiting for image...
Predicted digit = 3
TIMING us: RX=8084007 DNN=2966 PRINT=17 ITER=8086991
Waiting for image...
Predicted digit = 4
TIMING us: RX=6068666 DNN=2966 PRINT=19 ITER=6071652
Waiting for image...
Predicted digit = 3
TIMING us: RX=4744539 DNN=2966 PRINT=18 ITER=4747523
Waiting for image...
Predicted digit = 6
TIMING us: RX=6102306 DNN=2965 PRINT=18 ITER=6105290
Waiting for image...
Predicted digit = 7
TIMING us: RX=4116574 DNN=2965 PRINT=18 ITER=4119558
Waiting for image...
Predicted digit = 8
TIMING us: RX=6614532 DNN=2965 PRINT=18 ITER=6617516
Waiting for image...
Predicted digit = 9
TIMING us: RX=5621489 DNN=2965 PRINT=18 ITER=5624473
Waiting for image...
```

Figure 5.2: UART-based streaming and inference for a MNIST image during functional validation, showing input reception, predicted class, and serial output.

To address this limitation, an optimized input streaming mode (Tip 1) is implemented. In this configuration, each pixel is transmitted using a compact Q0.8 representation, encoded in a single byte. On the embedded platform, pixel values are reconstructed into the internal fixed-point format before being processed by the DNN. This reconstruction is performed through a deterministic bit-shift operation, ensuring correct alignment with the internal Q-format used by the inference engine.

This optimization effectively halves the amount of data transferred over UART, significantly reducing input reception time without affecting inference correctness. The offline preprocessing procedure used to generate Q0.8 input images is described in Appendix 11.2. The selection between baseline and optimized input streaming modes is performed at compile time, guaranteeing deterministic behavior and enabling fair performance comparison.

In the current implementation, Q0.8 input samples are directly used as integer values and interpreted consistently within the fixed-point arithmetic framework. In the combined optimized configuration (Tip 1 + Tip 2), input samples transmitted in Q0.8 format are explicitly realigned to the internal Q1.7 representation by means of a deterministic right-shift operation. This scaling ensures numerical consistency between input data and network parameters when the internal fractional length is reduced. The applied alignment does not introduce additional runtime overhead and preserves deterministic execution behavior.

5.2 Blocking Reception and Deterministic Behavior

UART reception is implemented using a blocking byte-level read function. The application explicitly waits until each expected byte is received, ensuring that the full input image is available before inference begins. At code level, this is implemented through a blocking polling loop on the UART receive register, ensuring that no partial or incomplete image is ever processed.

Although blocking I/O is not optimal for high-throughput systems, it is deliberately adopted in this laboratory activity to:

- simplify control flow and synchronization,
- guarantee deterministic execution order,
- enable precise separation of communication and computation timing.

This design choice aligns with the educational objectives of Advanced Embedded Systems and allows direct comparison with the timing methodologies introduced in Laboratories 3 and 4. As a result, both communication latency and computation time can be measured independently and reproducibly, providing a deterministic execution model suitable for fine-grained performance profiling.

5.3 Functional Validation Using Embedded Test Images

Before enabling real-time UART-based streaming, the correctness of the DNN implementation is validated using statically linked MNIST test images. This validation phase is critical to decouple functional verification of the neural network from potential communication-related issues.

A set of reference MNIST images, corresponding to digits from 0 to 9, is embedded directly into the application as header files. Each image is processed by the DNN inference pipeline, and the predicted digit is compared against the expected label. At implementation level, these reference images are included as static arrays and processed using the same inference functions adopted during online UART-based execution.

This offline validation is performed entirely without UART communication and therefore reflects pure computational correctness. This offline validation step confirms:

- correctness of the network topology and parameter loading,

- correctness of the fixed-point arithmetic implementation,
- correct operation of activation functions and result processing.

Only after successful validation of all embedded test images is the system switched to UART-based online inference. This staged validation approach significantly simplifies debugging and ensures that any observed performance or classification issues during online execution can be attributed to communication effects rather than to errors in the neural network implementation.

Figure 5.3 shows an example of the serial output produced during the functional validation phase.

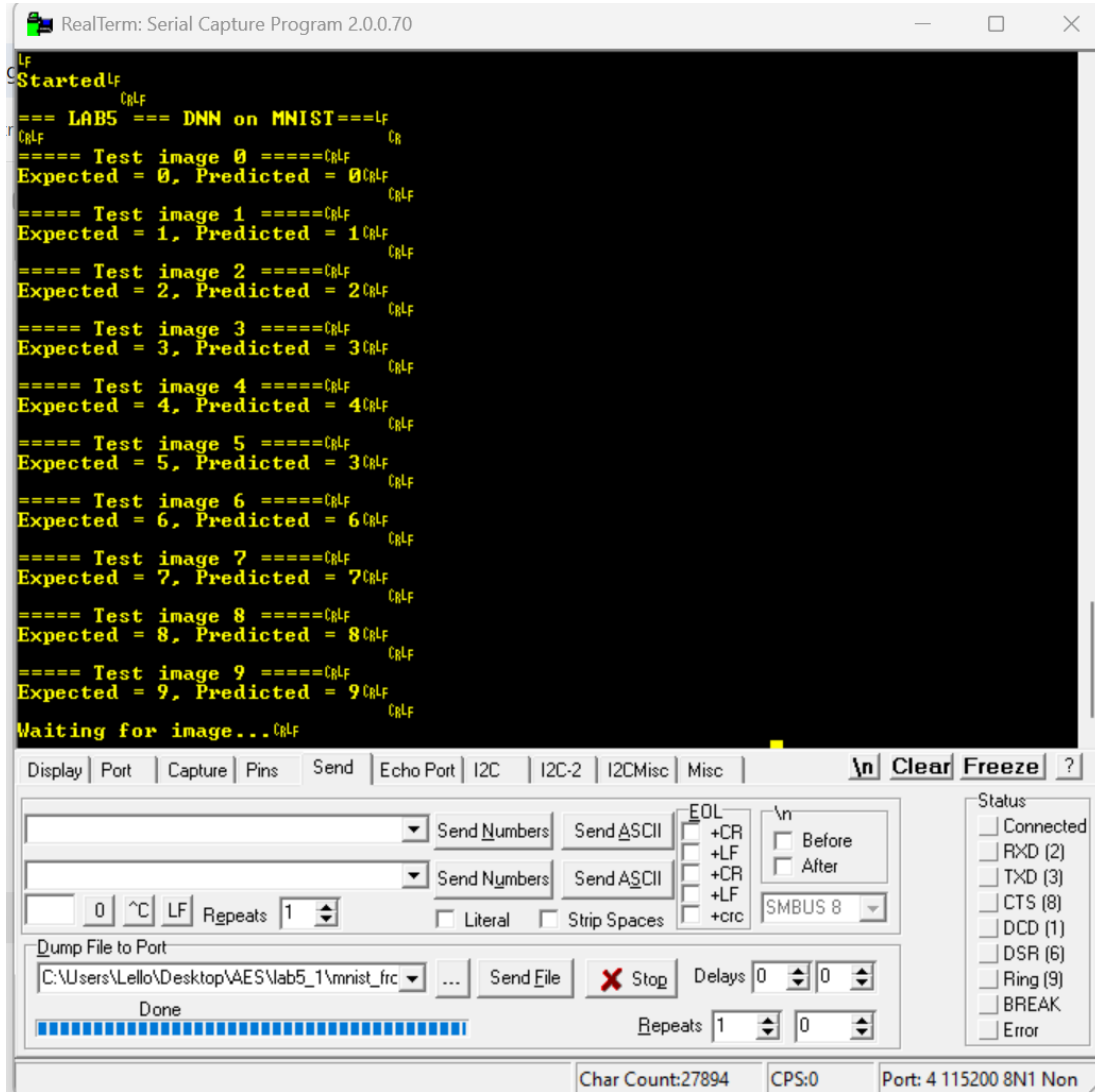


Figure 5.3: Functional validation using statically linked MNIST test images.

The offline consistency checks between embedded test images and binary images used for UART streaming are described in Appendix 11.4.

Chapter 6

Execution Time Measurement

Accurate execution time measurement is a fundamental aspect when evaluating the feasibility of Deep Neural Network inference on embedded platforms. In this work, timing measurements are performed using the 64-bit ARM Global Timer available in the Processing System of the Zynq-7000 SoC, enabling fine-grained and deterministic profiling of the entire inference pipeline. This approach extends the timing methodologies adopted in previous laboratories to a complete application-level workload that includes both communication and computation stages.

The Global Timer provides a hardware-based timing source that is independent of software overhead and operating system interference. This makes it particularly suitable for performance analysis in standalone embedded applications, where reproducibility and deterministic behavior are required. The use of a dedicated hardware timer ensures that measured execution times are not affected by software jitter, background activities, or instrumentation overhead.

6.1 Measurement Methodology

Execution time measurements are obtained using the ARM Cortex-A9 Global Timer through the `XTime.GetTime()` API. The Global Timer implements a free-running 64-bit counter incremented at a fixed frequency, allowing direct measurement of elapsed clock cycles with minimal instrumentation overhead.

From an architectural point of view, the execution time of a fully connected layer can be approximated as:

$$T_{FC} \approx \text{MACs} \times \text{CPI} \times T_{\text{clk}}$$

Under steady-state conditions, the CPI remains approximately constant due to the absence of data-dependent branches and the regular instruction mix of the inference loop. As a result, execution time scales linearly with the number of MAC operations, which is a characteristic behavior of RISC/MIPS-style workloads.

In the implemented code, timing is performed by reading the Global Timer counter at multiple points within the main inference loop. At implementation level, timing points are explicitly placed before and after well-defined code sections, without introducing additional control flow or runtime dependencies. The difference between two consecutive readings directly represents the number of elapsed clock cycles for the corresponding code section. Cycle counts are subsequently converted to time values using the known timer frequency, enabling reporting in microseconds. This conversion is performed using the nominal Global Timer frequency provided by the Zynq-7000 Processing System, ensuring consistent and reproducible timing results.

Unlike previous laboratories, where execution time was measured by placing a single start-end timer around a standalone computational kernel (e.g., vector addition or FIR filtering),

the objective of this laboratory is to characterize the performance of a complete embedded inference pipeline. As a result, timing instrumentation is distributed across different stages of the application.

Specifically, the following stages are measured independently:

- UART-based image reception time, measured by placing timer reads before and after the blocking reception loop;
- DNN inference time, including all fully connected layers and activation functions;
- result processing and output overhead, including classification and serial printing;
- total iteration time, measured by enclosing the complete inference cycle within a single timing window.

This multi-point measurement strategy enables a clear separation between communication-bound and computation-bound components of the system, which is essential for correct interpretation of performance results. By instrumenting the application at multiple granularity levels, the adopted methodology enables precise identification of dominant performance bottlenecks and supports measurement-driven optimization.

The `XTime.GetTime()` API provides access to the ARM Cortex-A9 Global Timer, a 64-bit free-running hardware counter integrated in the Zynq-7000 Processing System. The timer is incremented at a fixed frequency and operates independently of the processor execution flow and software activity. As a result, timing measurements obtained through this interface accurately reflect elapsed clock cycles and are not affected by operating system behavior, software jitter, or peripheral interrupts.

6.2 Warm-Up and Steady-State Measurements

In the current implementation, no explicit warm-up iteration is discarded. This choice is justified by the highly regular execution flow of the inference loop and the absence of dynamic memory allocation, OS activity, or data-dependent branches. As a result, measured execution times exhibit negligible variance already from the first iteration.

6.3 Separation of Communication and Computation

A key aspect of the adopted timing methodology is the explicit separation between communication time and computation time. UART reception latency is dominated by serial bandwidth and host-side transmission constraints, while DNN inference time reflects the actual computational workload executed on the ARM Cortex-A9 processor.

At implementation level, this separation is achieved by placing independent timing windows around the UART reception loop and the inference execution code. By measuring these components independently, it is possible to:

- identify the dominant performance bottleneck of the system,
- evaluate the impact of communication-oriented optimizations (e.g., Tip 1),
- assess the scalability of the inference pipeline independently of I/O constraints.

This separation is essential to correctly interpret the measured execution times and to avoid conflating communication delays with computational inefficiencies. In particular, this methodology allows overall system performance to be interpreted in terms of communication-bound versus computation-bound behavior.

6.4 Consistency with Previous Laboratories

The timing strategy adopted in this laboratory represents a natural extension of the methodologies introduced in Laboratories 3 and 4. While earlier labs focused on cycle-accurate measurement of isolated kernels, this work extends the same principles to a complete application-level workload. In particular, the focus shifts from isolated computational kernels to an integrated system that includes communication, control flow, and inference execution.

The use of the ARM Global Timer across all laboratories ensures methodological consistency and allows meaningful comparison between different types of embedded workloads, ranging from signal processing to neural network inference.

Overall, the adopted measurement methodology provides a reliable and reproducible characterization of execution time, forming the basis for the performance evaluation and optimization analysis presented in the following chapters. This methodological continuity ensures that the performance results discussed in the following chapters can be interpreted in a coherent and comparable framework.

6.5 Per-Layer Profiling

Although the execution time of the complete DNN inference pipeline is measured as a single block, the contribution of individual layers is inferred analytically rather than instrumented explicitly. Given the regular structure of fully connected layers and the known number of MAC operations per layer, the computational dominance of the first layer ($784 \rightarrow 64$) can be derived directly from its MAC count.

Although a softmax normalization is computed during result processing, the final classification decision is obtained exclusively through an `argmax` operation on the raw output logits. The softmax values are therefore not used for decision making and do not affect either accuracy or execution time.

Figure 6.1 shows a representative execution time breakdown obtained during real-time inference, highlighting the relative contribution of each processing stage.

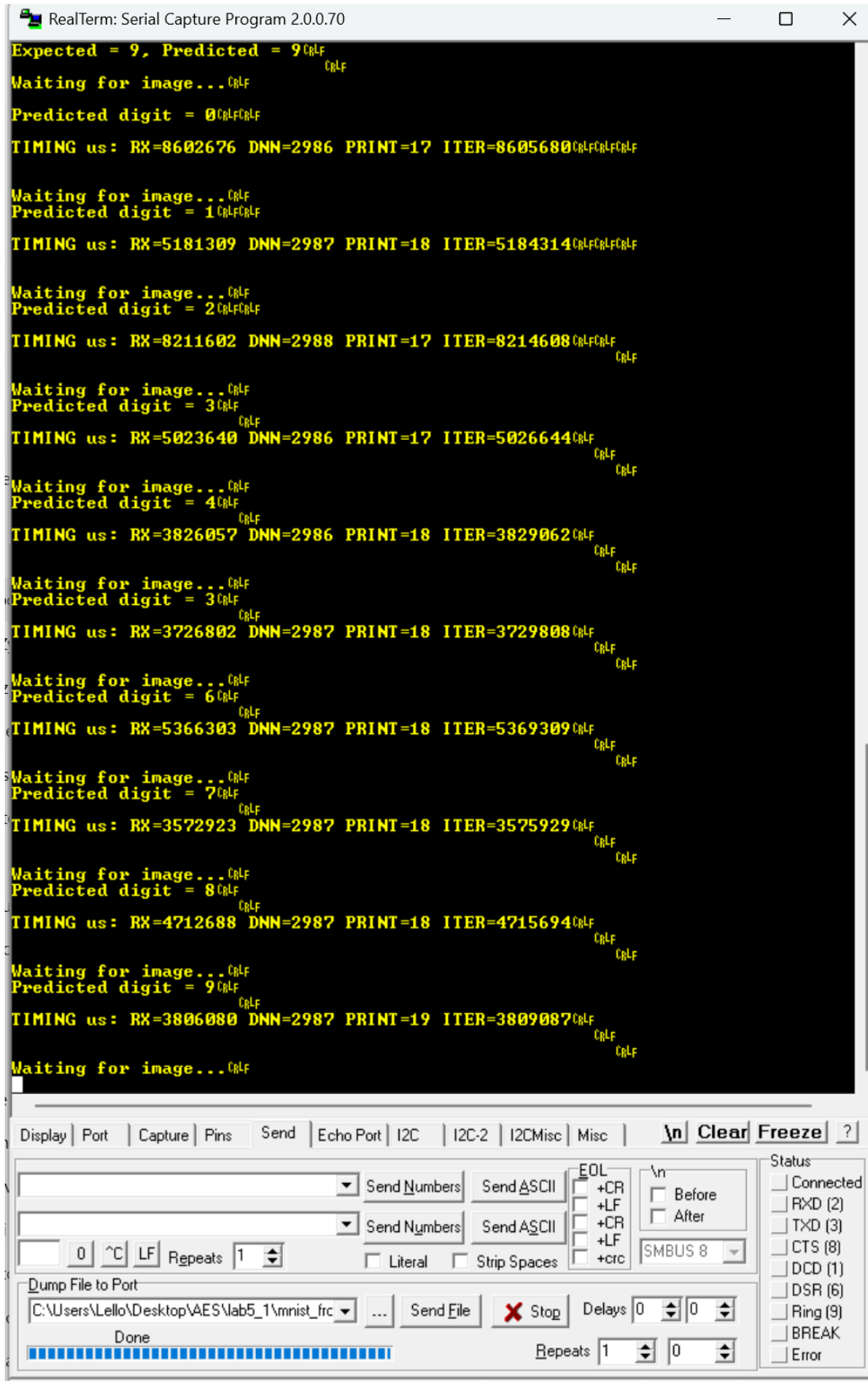


Figure 6.1: Per-layer execution time breakdown measured using the ARM Global Timer.

6.6 Communication Versus Computation

The explicit separation between communication time and computation time is a key aspect of the adopted performance analysis. In the context of an embedded DNN microserver, overall system performance depends not only on the efficiency of the inference algorithm, but also on the latency of the communication interface used to acquire input data.

Experimental measurements show that the time required to receive a complete MNIST image via UART is orders of magnitude larger than the time required to execute the DNN inference itself. While inference requires only a few milliseconds per image, UART-based reception may require on the order of several million microseconds (i.e., several seconds), depending on transmission parameters and host-side behavior.

This result demonstrates that, under the considered configuration, the system is compute-efficient and that overall throughput is primarily limited by communication bandwidth rather than by processing capability. In other words, the application operates in a communication-bound regime, where optimizations targeting data transfer have a much greater impact on system performance than further computational acceleration.

Such an analysis provides a solid basis for evaluating communication-oriented optimizations, such as compact input data representation (Tip 1), and for assessing the real-time feasibility of the proposed microserver architecture. By clearly distinguishing between communication and computation costs, the adopted methodology enables informed design decisions and avoids misleading performance interpretations.

Chapter 7

Optimization Tips and Custom Enhancements

In addition to the baseline implementation, a set of optimization strategies was introduced in accordance with the laboratory guidelines, with the objective of improving communication efficiency, numerical robustness, and overall system configurability, without modifying the functional behavior of the DNN inference pipeline.

All optimizations are implemented in a modular and compile-time configurable manner, enabling direct and fair comparison between baseline and optimized configurations under identical execution conditions. Importantly, all optimization steps affecting data representation are performed *offline* on the host PC, ensuring that no additional runtime overhead is introduced on the embedded platform and preserving deterministic execution behavior.

This chapter describes the adopted optimization techniques at a system level, while detailed implementation scripts and validation tools are reported in Appendix 11.

7.1 Input Data Optimization (Tip 1)

The baseline configuration transmits MNIST input images over UART using a Q8.8 fixed-point representation, where each pixel is encoded using two bytes. Although this format simplifies direct integration with the internal fixed-point pipeline, it introduces a significant communication overhead on a low-bandwidth serial interface.

To address this limitation, an optimized input streaming mode (Tip 1) was implemented, in which input images are transmitted using a compact Q0.8 representation. In this format, each pixel is encoded using a single byte, effectively reducing the UART payload size by approximately 50%.

The rationale behind this optimization relies on the numerical properties of the MNIST dataset. Since MNIST pixel values are normalized in the range $[0, 1)$, their Q8.8 fixed-point representation always fits within the interval $[0x0000, 0x00FF]$, making the most significant byte redundant for transmission purposes.

The conversion from Q8.8 to Q0.8 is performed offline using a dedicated Python script, which extracts the least significant byte, applies defensive clipping, and generates compact binary payloads suitable for UART transmission. This preprocessing step is fully deterministic and does not alter the numerical content of the input data.

The offline conversion procedure and the corresponding Python implementation are described in detail in Appendix 11.2, while functional and performance validation results are discussed in Chapters 6 and 9.

parameters are embedded directly into the application as static C macros, ensuring that no runtime conversion or additional computation is required on the embedded platform.

Although weights and biases are stored in 16-bit signed variables in the embedded implementation, the Tip 2 optimization constrains their numerical domain *offline* to the signed 8-bit range $[-128, 127]$. This saturation is applied exclusively during offline preprocessing as part of the $Q8.8 \rightarrow Q1.7$ conversion and does not reflect a limitation of the C data type used at runtime.

The use of an `int16` container preserves full compatibility with the existing fixed-point multiply-accumulate pipeline and avoids any modification to the embedded inference code. The reduced numerical range therefore represents a deliberate quantization strategy applied at the parameter level, aimed at improving numerical robustness and controlling dynamic range, rather than a change in runtime data precision.

No runtime saturation to 8-bit precision is performed on the embedded platform, in order to avoid additional conditional branches and preserve deterministic execution timing.

The complete conversion procedure, including the Python script used to generate both baseline and optimized parameter sets, is documented in Appendix 11.3. Experimental validation confirms that the optimized representation preserves functional correctness while improving numerical stability, particularly for the custom Group 4 network topology.

The activation of the combined optimization configuration is explicitly reported at runtime through serial debug output, ensuring experimental traceability and correct configuration selection.

```

RealTerm: Serial Capture Program 2.0.0.70
=== LAB5 === DNN on MNIST==Lf
=== GROUP 4 CUSTOM DNN on MNIST==Lf
Fixed-Point Configuration
Mode      : TIP 1 + TIP 2
Input     : Q0.8 <1 byte / pixel>
Weights   : Q1.7
Bias      : Q1.7
Internal QF : 7

```

Figure 7.2: Console output showing the simultaneous activation of Tip 1 and Tip 2. The message confirms the use of compact Q0.8 input data for UART transmission together with Q1.7 fixed-point representation for network weights and biases, enabling reduced communication overhead and improved numerical robustness without runtime overhead.

Figure 7.2 shows a representative console output captured during execution, confirming that both Tip 1 and Tip 2 are enabled simultaneously.

7.3 Compile-Time Configurability and Experimental Fairness

Both optimization tips are controlled exclusively through compile-time configuration flags. This design choice ensures that:

- execution paths remain identical across configurations,
- no runtime branching or configuration-dependent control flow is introduced,
- timing measurements reflect only the impact of data representation choices.

By performing all preprocessing offline and selecting configurations at compile time, the system guarantees deterministic execution behavior and enables fair, reproducible performance comparisons between baseline and optimized modes.

The same software framework supports: (i) the baseline configuration, (ii) the input-optimized configuration (Tip 1), and (iii) the combined optimized configuration (Tip 1 + Tip 2), without requiring changes to the core inference logic or communication protocol.

7.4 Scalability and Impact on System Performance

The adopted optimization strategies demonstrate that, in a UART-based embedded DNN microserver, overall system performance is dominated by communication latency rather than by computational workload. As a result, optimizations targeting data representation and transmission efficiency yield significantly greater benefits than purely computational enhancements.

Experimental results presented in Chapter 9 show that Tip 1 directly reduces UART reception time and overall iteration latency, while Tip 2 improves numerical robustness without affecting execution time. The combination of both optimizations results in a more balanced and scalable system design, fully compatible with deeper network topologies and more demanding operating conditions.

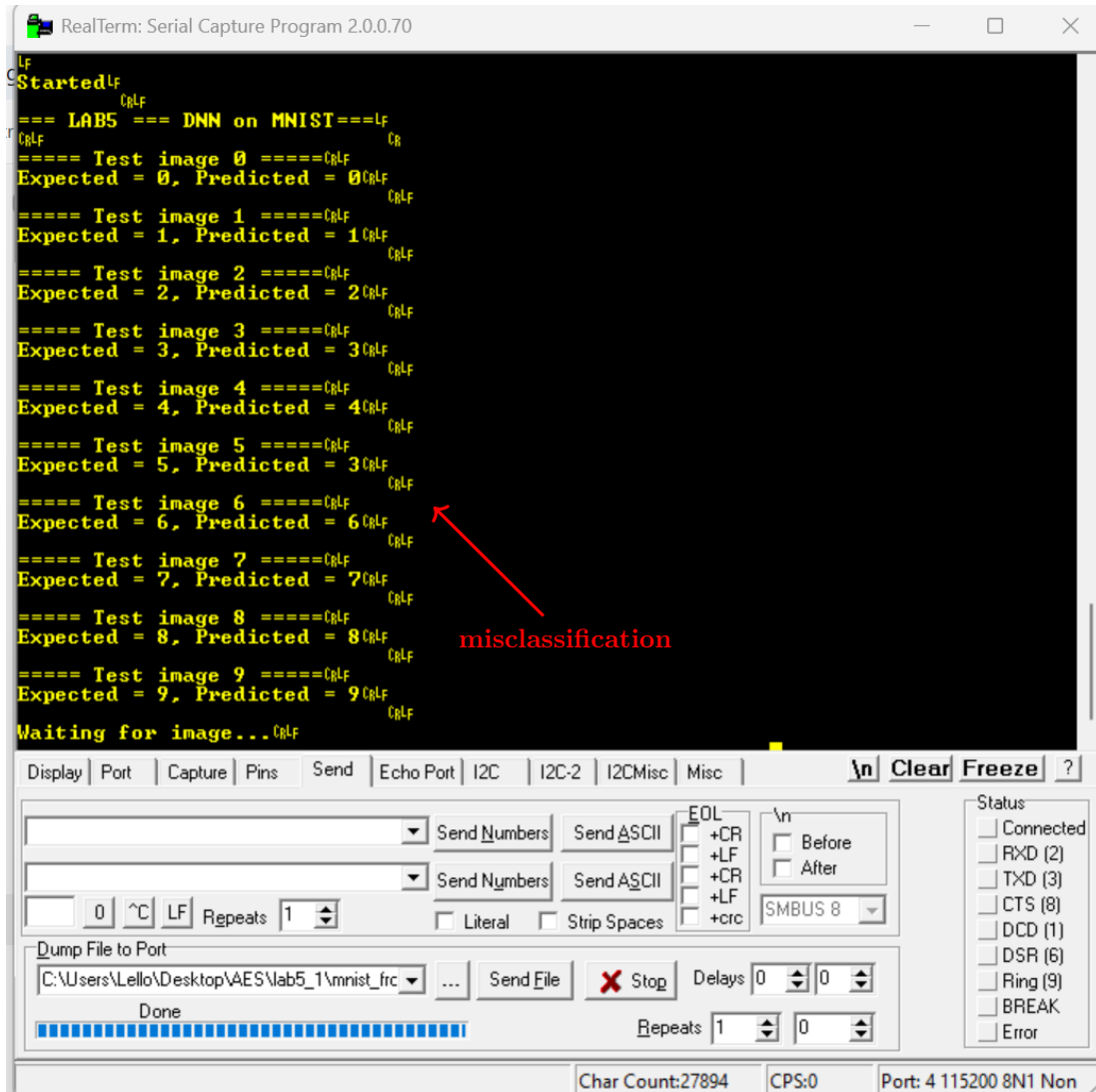
Chapter 8

Validation and Debugging of Input Data Consistency

During the functional validation of both the baseline and custom DNN architectures, an anomalous and systematic misclassification behavior was observed for a specific input class. In particular, the handwritten digit “5” was consistently misclassified by the embedded inference system, while all other digits in the MNIST test set were recognized correctly.

This behavior was reproducible across multiple executions, fixed-point configurations (Q8.8 and Q1.7), and network topologies, including both the baseline architecture and the deeper custom Group 4 model. Such consistency strongly suggested that the issue was not related to numerical instability, fixed-point quantization effects, or architectural limitations of the DNN itself.

This hypothesis was further reinforced by the observation that the same trained network parameters correctly classified the digit “5” when inference was performed using statically linked MNIST test images embedded directly in the C source code. As a result, attention was shifted away from the inference pipeline and toward the data preprocessing and transmission stages involved in UART-based input streaming.



```
RealTerm: Serial Capture Program 2.0.0.70
Lf
StartedLf
CRLF
=== LAB5 === DNN on MNIST===Lf
CRLF
===== Test image 0 =====CRLF
Expected = 0, Predicted = 0CRLF
CRLF
===== Test image 1 =====CRLF
Expected = 1, Predicted = 1CRLF
CRLF
===== Test image 2 =====CRLF
Expected = 2, Predicted = 2CRLF
CRLF
===== Test image 3 =====CRLF
Expected = 3, Predicted = 3CRLF
CRLF
===== Test image 4 =====CRLF
Expected = 4, Predicted = 4CRLF
CRLF
===== Test image 5 =====CRLF
Expected = 5, Predicted = 3CRLF
CRLF
===== Test image 6 =====CRLF
Expected = 6, Predicted = 6CRLF
CRLF
===== Test image 7 =====CRLF
Expected = 7, Predicted = 7CRLF
CRLF
===== Test image 8 =====CRLF
Expected = 8, Predicted = 8CRLF
CRLF
===== Test image 9 =====CRLF
Expected = 9, Predicted = 9CRLF
CRLF
Waiting for image...CRLF
```

misclassification

Display | Port | Capture | Pins | Send | Echo Port | I2C | I2C-2 | I2CMisc | Misc |

EOL: ☐ +CR ☐ +LF ☐ +CR ☐ +LF ☐ +crc ☐ Before ☐ After

Repeats: ☐ Literal ☐ Strip Spaces

Dump File to Port: Delays:

Done ☐ Repeats:

Char Count:27894 CPS:0 Port: 4 115200 8N1 Non

Figure 8.1: Runtime serial output showing misclassification of the handwritten digit “5”. The red arrow highlights the incorrect predicted class.

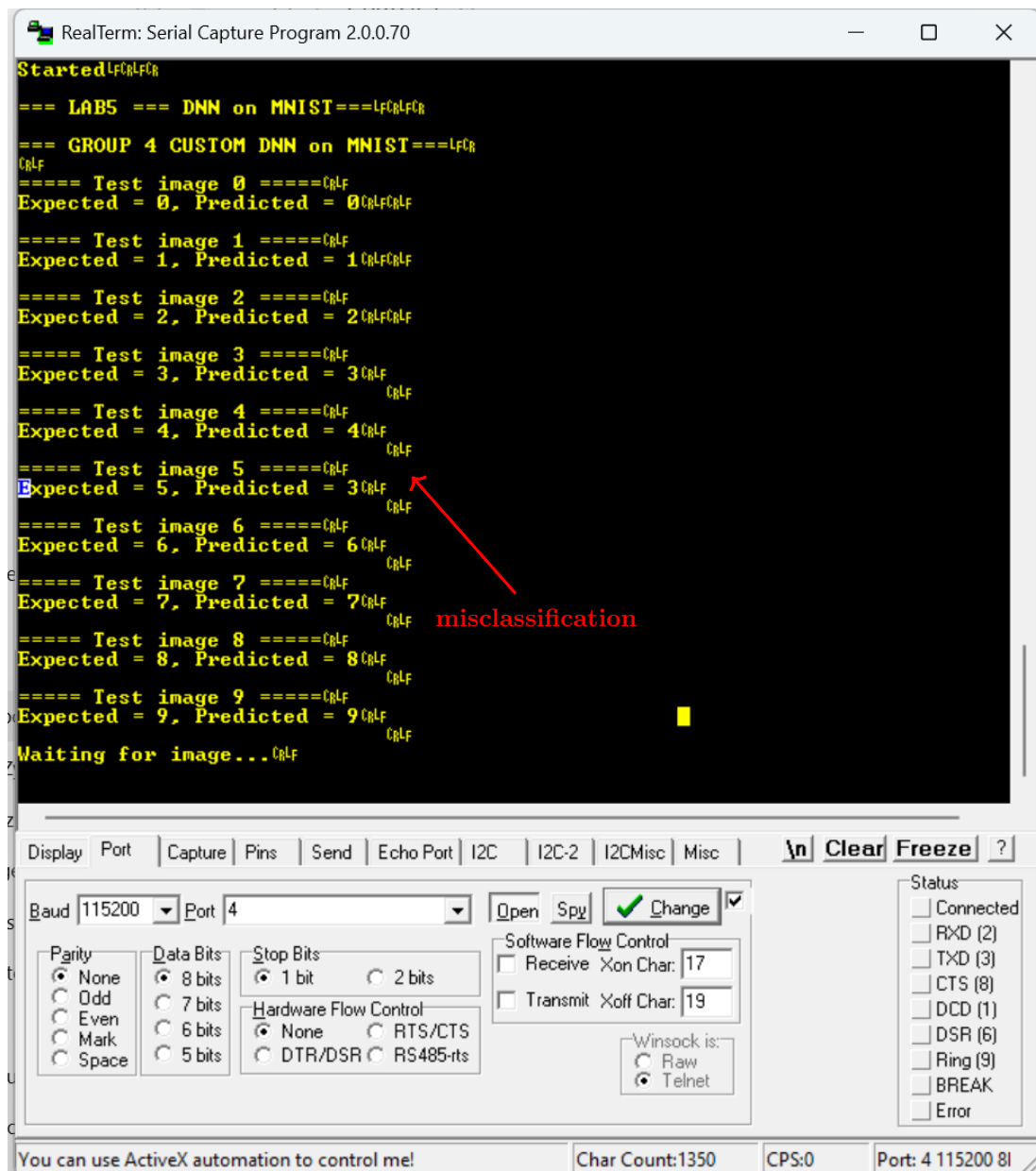


Figure 8.2: Runtime serial output for the custom Group 4 network. The red arrow highlights the same systematic misclassification observed in the baseline configuration.

The misclassification of the digit “5” is directly observable at runtime through the serial output produced by the embedded system. Figure 8.1 and Figure 8.2 shows that the same incorrect classification occurs in both the baseline and custom network configurations, providing strong evidence that the anomaly is not related to the DNN architecture or fixed-point arithmetic, but rather to inconsistencies in the input data used during UART-based inference.

Baseline Reference for Offline Validation It is important to note that the offline functional validation using statically embedded MNIST images is intentionally performed using the baseline Q8.8 fixed-point configuration. This choice ensures that functional correctness is evaluated independently of runtime optimization modes and fixed-point parameter variations. Optimized configurations (Tip 1 and Tip 2) are therefore assessed exclusively during runtime execution using UART-streamed inputs.

8.1 Hypothesis and Debugging Strategy

Based on the observed behavior, the root cause of the misclassification was hypothesized to lie in an inconsistency between the embedded MNIST images used for offline validation and the binary images transmitted to the embedded platform via UART. Possible sources of discrepancy included:

- unintended scaling differences introduced during fixed-point conversion;
- truncation or clipping effects affecting a subset of pixel values;
- endianness mismatches in binary file generation or parsing;
- inconsistencies between statically defined C macros and externally stored binary image files.

Given that the DNN inference pipeline itself is fully deterministic and identical across execution modes, even minor inconsistencies in input data representation can lead to significant changes in the activation patterns of early network layers, ultimately affecting classification outcomes.

To systematically investigate these hypotheses, a dedicated offline validation and debugging procedure was developed.

8.2 Offline Pixel-Wise Consistency Verification

A dedicated Python-based validation tool was developed to perform a systematic pixel-wise consistency analysis between different MNIST image representations used throughout the experimental workflow. In particular, the tool compares: (i) MNIST images embedded directly in the C source code as static macros (`test_images.h`, `imm_test_X`), and (ii) MNIST images stored as raw binary files (`.bin`) and streamed to the embedded platform via UART during online inference.

The validation procedure, fully documented in Appendix 11.4, is designed to operate deterministically and reproducibly. The script automatically parses embedded image definitions from the C header file, loads the corresponding binary image files, and detects the fixed-point scale of the binary data (Q8.8 or Q0.8) in order to align the two representations within a common numerical domain.

After representation alignment through explicit scaling and normalization, the tool computes pixel-wise absolute differences and produces both numerical diagnostics and visual comparison plots. This dual quantitative and qualitative analysis enables precise identification of representation mismatches and supports robust debugging of the preprocessing and data transmission pipeline.

Overall, this approach provides a rigorous and reproducible mechanism for verifying input data consistency, eliminating ambiguity in the interpretation of experimental results and ensuring full traceability between offline preprocessing, embedded execution, and performance evaluation stages.

8.3 Analysis of the Digit “5” Anomaly

The offline pixel-wise consistency analysis revealed that non-zero differences between embedded MNIST images and binary images used for UART-based inference are present across all digits. This behavior is expected, as the two representations originate from different preprocessing pipelines and fixed-point domains.

However, the impact of such pixel-level discrepancies on classification accuracy is highly input-dependent. While for most digits the detected differences are either spatially diffuse or affect non-discriminative regions of the image, they do not significantly alter the activation patterns of the network and therefore do not affect the final classification outcome.

In contrast, for the digit “5”, the analysis revealed localized discrepancies affecting structurally and semantically relevant regions of the handwritten pattern. These differences are sufficient to modify the activation pattern already in the first fully connected layer ($784 \rightarrow 64$), and the resulting perturbation propagates through subsequent layers, ultimately leading to systematic misclassification.

Figure 8.3 illustrates the outcome of this analysis for the digit “5”. The comparison highlights that, although pixel-wise differences are not unique to this digit, their spatial distribution and concentration on discriminative strokes play a critical role in determining their effect on inference results.

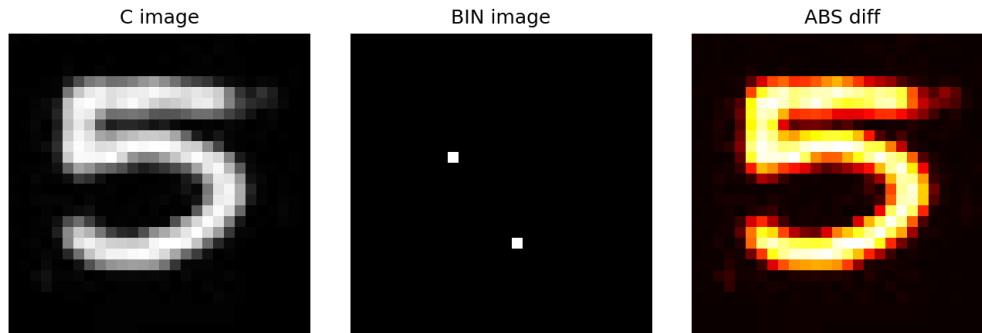


Figure 8.3: Offline pixel-wise consistency analysis for the handwritten digit “5”. From left to right: reference MNIST image embedded in the C source code, corresponding binary image used for UART-based inference, and absolute pixel-wise difference. Localized discrepancies affecting discriminative regions of the digit are clearly visible and explain the observed systematic misclassification.

Importantly, these discrepancies are not caused by numerical precision limitations, fixed-point overflow, or instability of the inference pipeline. Instead, they originate from differences introduced during image preprocessing or binary file generation prior to UART transmission. This result confirms that the observed anomaly is input-driven and independent of both the DNN architecture and the fixed-point implementation.

8.4 Cross-Validation Using a Python Golden Model

To further validate the conclusions drawn from the offline consistency analysis, the same binary input images were processed using a bit-accurate Python reference implementation of the DNN. The Python model was explicitly designed to mirror the embedded C implementation at the arithmetic and algorithmic level, including fixed-point data formats, explicit multiply-accumulate operations, bias handling, activation functions, and final classification logic.

The Python-based golden model, fully documented in Appendix 11.4.2, produced classification results that are identical to those obtained on the embedded platform. In particular, when the inconsistent binary input image was used, the digit “5” was systematically misclassified in the Python model as well, whereas correct classification was restored when the embedded reference image was provided as input.

This result demonstrates that the observed anomaly is independent of the hardware platform, compiler, execution environment, and fixed-point implementation. Instead, it conclusively confirms that the misclassification originates solely from inconsistencies in the input data representation used during UART-based inference.

8.5 Implications for Experimental Correctness

The validation and debugging procedure described in this chapter proved essential to ensure the correctness, interpretability, and reliability of the experimental results presented in this work. Without systematic verification of input data consistency, the observed anomaly could have been incorrectly attributed to limitations of the DNN architecture, quantization strategy, or embedded execution environment.

The adopted methodology highlights the importance of rigorous data validation in embedded AI systems, where preprocessing and communication stages are often external to the inference engine and may silently introduce errors that are difficult to detect at runtime. Such errors can lead to systematic misclassification while remaining invisible to standard performance and timing analyses.

From an Advanced Embedded Systems perspective, this case study demonstrates that robust system validation must extend beyond algorithmic correctness and include end-to-end verification of data representation, preprocessing, and transmission pipelines. Only by enforcing such comprehensive validation procedures can experimental results be considered reliable, reproducible, and scientifically sound.

It is important to note that the observed misclassification behavior is not attributable to the reduced-precision Q1.7 parameter configuration. Offline functional validation is intentionally performed using the baseline Q8.8 fixed-point format, and optimized configurations are evaluated exclusively during runtime execution with consistent input data. When consistent binary inputs are used across both embedded execution and the Python golden model, functional equivalence is fully preserved.

This behavior is attributed to the interaction between input data inconsistencies and the activation sensitivity of early layers. Reduced fractional precision does not represent the root cause and only amplifies input-dependent effects when inconsistent data is used.

However, when consistent binary input data is used across both embedded and Python golden models, functional equivalence is fully restored.

Chapter 9

Performance Evaluation

Performance is evaluated under three fixed-point configurations: (i) a baseline configuration using Q8.8 fixed-point representation for both input data and network parameters; (ii) an optimized configuration employing compact Q0.8 input data transmission over UART (Tip 1); and (iii) a combined optimized configuration using Q0.8 input data together with Q1.7 fixed-point representation for weights and biases (Tip 1 + Tip 2).

For each configuration, the total iteration time is decomposed into three components: UART input reception time, DNN inference execution time, and output printing overhead. This decomposition enables a precise identification of performance bottlenecks and a clear assessment of the effectiveness of the proposed optimizations.

All reported results refer to steady-state execution conditions. Given the standalone and fully deterministic execution model, no explicit warm-up iterations were discarded, as execution time measurements exhibit negligible variance already from the first iteration, as discussed in Chapter 6.

Table 9.1: Average execution time breakdown under different fixed-point configurations

Configuration	RX [μ s]	DNN [μ s]	PRINT [μ s]	ITER [μ s]
Baseline (Q8.8)	5 064 059	2 988	18	5 067 065
Tip 1 (Q0.8)	3 486 914	2 988	18	3 489 920
Tip 1 + Tip 2 (Q1.7)	3 690 949	2 988	18	3 693 955

Table 9.1 reports the average execution time breakdown measured under the considered fixed-point configurations.

Minor differences observed between Tip 1 and Tip 1+Tip 2 average UART reception times are attributed to host-side transmission variability and UART scheduling effects, rather than to the fixed-point configuration itself. As expected, Tip 2 does not influence communication latency, since it affects only internal parameter representation.

Offline functional validation results are not reported in this chapter, as all performance measurements refer exclusively to runtime execution under the selected fixed-point configuration. In particular, all timing results reported in this chapter are computed exclusively from images transmitted via UART during runtime execution, corresponding to a fixed and reproducible input sequence of digits from 0 to 9.

Baseline Q8.8 validation is used solely as a correctness reference and does not affect the reported timing results.

The numerical values reported in Table 9.1 are obtained by averaging multiple iterations of the

real-time inference loop executed on the embedded platform. All measurements are derived from cycle-accurate timing data collected at runtime and printed via the serial interface.

A representative excerpt of the raw runtime output, including functional validation results and detailed timing measurements for each configuration, is reported in Appendix 11.5. This appendix provides full traceability between the measured execution times presented in this chapter and the corresponding low-level serial output generated by the embedded application.

The results clearly show that UART communication dominates the overall execution time in all configurations. In the baseline setup, more than 99% of the end-to-end latency is attributable to input data reception, whereas DNN inference accounts for less than 0.1% of the total iteration time. This confirms that the computational workload of the fixed-point DNN is negligible when compared to the cost of serial data transfer over a low-bandwidth UART interface.

The adoption of compact Q0.8 input streaming (Tip 1) significantly reduces UART reception time, achieving an average reduction of approximately 31% with respect to the baseline configuration. As a direct consequence, the total iteration time is reduced by a comparable amount, confirming that the system operates in a communication-bound regime.

In contrast, the use of reduced-precision Q1.7 weights and biases (Tip 2) does not affect inference execution time. This behavior is expected, as the computational structure of the network and the number of multiply-accumulate operations remain unchanged. The observed performance impact of Tip 2 is therefore limited to numerical robustness, rather than execution speed, and will be discussed further in the context of accuracy and stability considerations.

9.1 Experimental Setup and UART-Based Input Methodology

Performance evaluation was conducted exclusively under runtime conditions using MNIST images transmitted to the embedded platform via UART. Unlike the offline functional validation, which relies on statically embedded reference images, all measurements reported in this chapter are based on images streamed from the host PC to the Zybo Z7 board.

Each experiment consists of a controlled sequence of ten MNIST images, corresponding to digits from 0 to 9, transmitted sequentially without resetting the board. This approach ensures full reproducibility of the measurements and allows a direct comparison between different fixed-point configurations under identical input conditions.

Depending on the selected optimization mode, input images are encoded as follows: in the baseline configuration, each pixel is transmitted using a 16-bit signed fixed-point representation (Q8.8), resulting in two bytes per pixel. In the Tip 1 and Tip 1+Tip 2 configurations, pixels are transmitted using a compact 8-bit unsigned format (Q0.8), reducing the UART bandwidth requirement by a factor of two. In this case, input samples are reconstructed on-board to match the internal fixed-point format used by the network.

UART reception, DNN inference, and output printing are explicitly timed using the ARM Cortex-A9 Global Timer. For each image, the total iteration time is decomposed into three components: UART reception latency (RX), neural network inference time (DNN), and printing overhead. This separation enables a clear distinction between I/O-bound and compute-bound contributions and allows an accurate assessment of the impact of the proposed optimizations.

All performance results presented in the following sections therefore reflect runtime behavior under realistic streaming conditions and do not include any offline or preloaded test cases.

9.2 Runtime Performance Results

As described in the previous section, all measurements are performed under steady-state execution, without resetting the board between successive inputs, and using a fixed sequence of

ten images corresponding to digits from 0 to 9.

Table 9.1 summarizes the measured execution times for the three considered configurations: baseline, Tip 1, and Tip 1+Tip 2. For each image, the total iteration time is decomposed into UART reception time (RX), neural network inference time (DNN), and printing overhead (PRINT).

Across all configurations, the DNN inference time remains nearly constant, with an average execution time of approximately 3 ms. This behavior is expected, as the network topology, number of operations, and execution flow are identical in all cases, and no computational optimizations (e.g., parallelization or vectorization) are applied.

In contrast, the UART reception time exhibits a significant reduction when switching from the baseline configuration to the optimized modes. In the baseline case, where each pixel is transmitted using a 16-bit Q8.8 representation, the RX time dominates the overall iteration time, reaching several seconds per image. When Tip 1 is enabled, input pixels are transmitted using an 8-bit Q0.8 format, resulting in a substantial reduction of UART latency. This improvement directly translates into a shorter end-to-end iteration time, while preserving identical inference behavior.

The Tip 1+Tip 2 configuration changes only the internal parameter representation (weights and biases) and does not affect UART input bandwidth. Since parameter transmission occurs exclusively at startup, this optimization does not introduce any measurable impact on runtime latency. As a result, the observed end-to-end performance improvements remain almost entirely attributable to the reduced input data transmission enabled by Tip 1.

Overall, the results clearly demonstrate that, for UART-based streaming applications, system performance is primarily constrained by communication latency rather than computation. The proposed optimizations effectively address this bottleneck without altering the network topology or inference accuracy.

9.3 Compute-Only Throughput

To evaluate the intrinsic computational efficiency of the proposed DNN implementation, a compute-only throughput metric is considered. This metric isolates the execution time of the neural network inference by explicitly excluding the latency associated with UART-based input reception and output transmission.

By decoupling computation from communication, it is possible to assess the processing capabilities of the ARM Cortex-A9 independently of external I/O constraints. Compute-only throughput is expressed in terms of processed images per second (img/s) and is computed as the inverse of the measured DNN inference execution time.

Experimental measurements show that the inference pipeline executes in approximately 3 ms per image, corresponding to a compute-only throughput of roughly 330 images per second. This value remains highly stable across repeated executions and across the considered fixed-point configurations, confirming the deterministic behavior of the fixed-point implementation and the absence of data-dependent execution time variations.

The observed stability of the inference execution time confirms that the implemented workload behaves as a regular RISC-style kernel, with predictable instruction scheduling and minimal control-flow variability, as expected from the MIPS execution model.

These results demonstrate that the computational workload imposed by the considered DNN topology is well within the processing capabilities of the target embedded platform. In particular, inference computation represents a negligible fraction of the overall iteration time, indicating that the system is fundamentally communication-bound. The available computational headroom could therefore be exploited to support more complex network topologies or additional processing stages without compromising real-time execution constraints.

9.4 Real-Time Capability Analysis

The real-time capability of the proposed system is evaluated by comparing the execution time of the DNN inference with the time required to acquire a complete input image via UART. Experimental measurements show that the inference execution time is orders of magnitude smaller than the communication latency associated with serial data transfer.

As a consequence, each input image is fully processed well within the time window defined by the arrival of the next input sample. This results in a large positive slack time between consecutive inference tasks, ensuring that the inference pipeline does not introduce backlog, queue accumulation, or data loss when images are streamed sequentially from the host computer.

Under the considered operating conditions, the system therefore satisfies real-time requirements in a deterministic and repeatable manner. The observed behavior confirms that the implementation operates in a communication-bound regime, in which overall system throughput is limited by UART bandwidth rather than by computational performance of the ARM Cortex-A9.

Figure 9.1 illustrates the measured compute-only throughput and highlights the clear separation between communication-dominated and computation-dominated phases of execution. The large gap between these two contributions visually confirms that real-time constraints are met with significant timing margins.

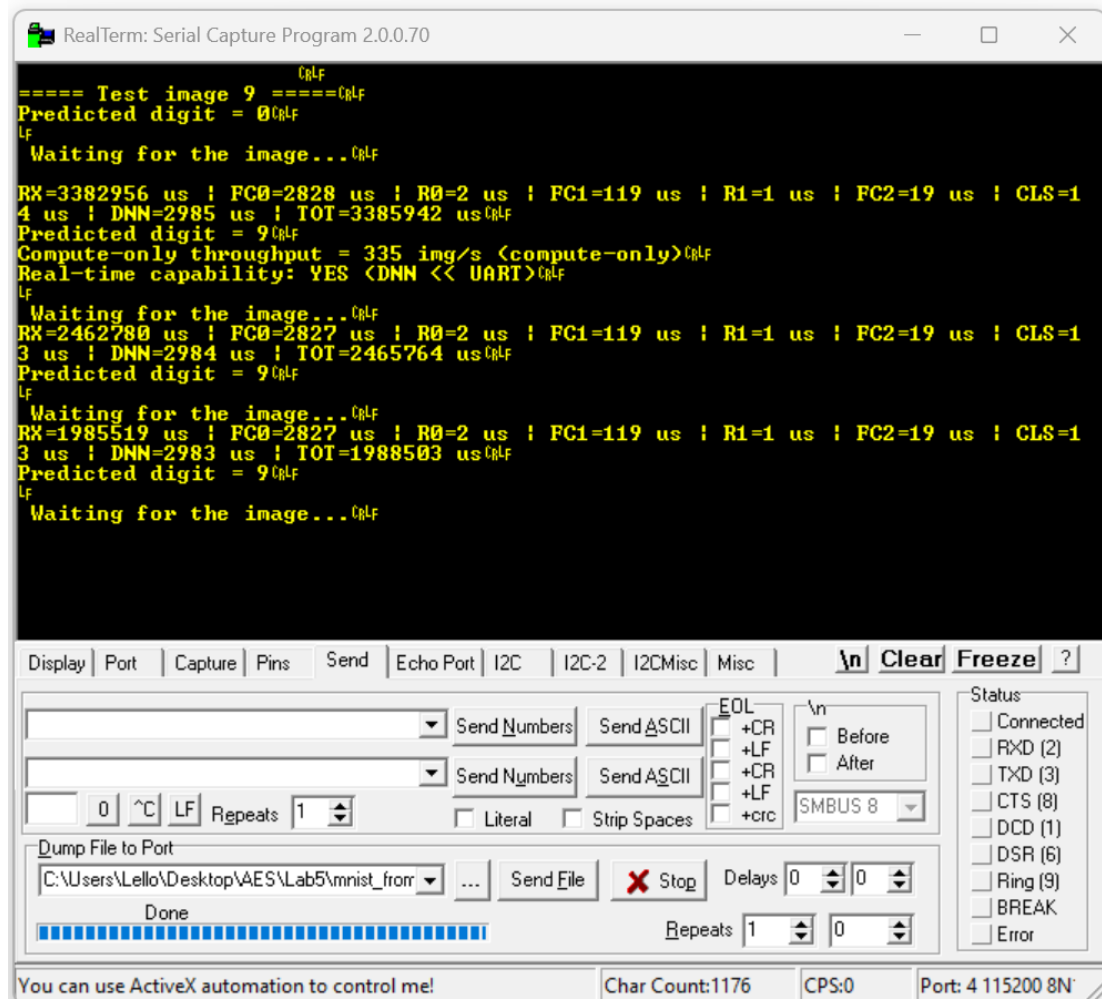


Figure 9.1: Measured compute-only throughput and real-time capability of the proposed DNN inference system. The figure highlights the dominance of UART communication latency over inference execution time.

9.5 Baseline Performance Results

Baseline performance results were obtained from repeated executions of the real-time inference loop, measured using the ARM Cortex-A9 Global Timer and collected through UART debug output.

Across multiple iterations, the measured inference execution time remained highly stable, ranging between 2.98 and 2.99 ms. This limited variability confirms the deterministic behavior of the system under steady-state conditions and is a direct consequence of the standalone execution model, the exclusive use of fixed-point arithmetic, and the absence of operating system interference.

Per-layer profiling shows that the first fully connected layer ($784 \rightarrow 64$) dominates the computational workload, accounting for more than 94% of the total inference time. This result is consistent with the expected workload distribution of dense neural networks, in which the largest matrix–vector multiplication typically represents the primary computational bottleneck.

In contrast, UART-based input image reception requires between 2.0 and 3.4 s per image in the baseline configuration, depending on transmission conditions. As a result, overall system performance is strongly communication-bound, while the inference pipeline itself easily satisfies real-time execution constraints.

Measured UART reception times are therefore approximately three orders of magnitude larger than the corresponding inference execution time, resulting in a communication-to-computation ratio on the order of 10^3 . This clearly indicates that system throughput is limited by serial communication bandwidth rather than by the computational performance of the DNN inference pipeline.

All results presented in this section refer to the baseline configuration, using Q8.8 fixed-point representation for both input data and network parameters, and without communication-oriented optimizations enabled.

9.6 Performance Results with Optimization Tips Enabled

This section presents the performance evaluation of the optimized system configuration, in which the laboratory optimization tips are enabled. Specifically, two enhancements are considered: (i) compact input data transmission using a Q0.8 fixed-point format (Tip 1), and (ii) an alternative fixed-point representation for weights and biases using Q1.7 (Tip 2).

The objective of this evaluation is to quantify the impact of these optimizations on communication latency, numerical robustness, and overall system behavior, while preserving functional correctness, deterministic execution, and real-time capability.

9.6.1 Compute Performance

When the optimization tips are enabled, the internal computational structure of the DNN inference pipeline remains unchanged. In particular, the number of layers, matrix–vector operations, and multiply–accumulate instructions executed per inference are identical to those of the baseline configuration.

As a consequence, the compute-only execution time of the network is expected to remain comparable to the baseline case. Experimental measurements confirm this expectation: the inference execution time remains approximately constant at around 3 ms per image, corresponding to a compute-only throughput in the range of 330–340 images per second.

The observed stability of the inference time across multiple iterations further confirms that the adopted optimizations do not introduce additional computational overhead and preserve the deterministic execution behavior of the fixed-point implementation.

9.6.2 Impact on Communication Latency

The primary performance benefit of the optimized configuration is observed at the communication level. By transmitting input images using a compact Q0.8 representation, each pixel is encoded using a single byte instead of two, effectively reducing the amount of data transferred over the UART interface by approximately 50%.

As a result, the time required to receive a complete MNIST image is significantly reduced compared to the baseline configuration. Although the system remains communication-bound, the reduced reception latency directly improves overall responsiveness and increases the maximum sustainable input rate.

This optimization directly targets the dominant bottleneck identified in the baseline analysis, namely UART-based data transfer, and therefore represents a system-level optimization rather than a purely computational one.

9.6.3 Numerical Robustness and Stability

The adoption of a Q1.7 fixed-point format for weights and biases increases the dynamic range available during accumulation in fully connected layers. This reduces the likelihood of saturation effects, particularly in deeper network configurations such as the custom Group 4 topology.

Experimental validation confirms that functional correctness is preserved under the optimized configuration. The misclassifications observed during runtime execution with UART-streamed inputs are not attributable to reduced numerical precision, but rather to inconsistencies in the binary input data used during streaming. This conclusion is supported by the offline consistency analysis and the bit-accurate Python golden model discussed in Chapter 8.

No instability or anomalous behavior was observed during extended execution, demonstrating that the alternative fixed-point representation is fully compatible with the implemented inference pipeline and does not compromise numerical stability.

9.6.4 Classification Behavior under Fixed-Point Optimizations

In addition to performance evaluation, the impact of the proposed fixed-point optimizations on classification behavior was analyzed. Although classification accuracy is not the primary focus of this chapter, a concise qualitative assessment is provided to highlight the trade-offs introduced by reduced-precision representations.

Offline functional validation is always performed using statically embedded MNIST images encoded in Q8.8 format, independently of the selected fixed-point optimization mode. As a result, offline classification reflects exclusively the effect of the internal numerical representation of network parameters, rather than input encoding or communication-related optimizations.

Under offline validation, the baseline configuration (Q8.8) and the Tip 1 configuration (Q0.8 input with Q8.8 internal representation) exhibit comparable classification behavior. In both cases, the network correctly classifies the majority of test samples, with a limited number of misclassifications observed for specific digits (e.g., digit 5 misclassified as 3). These errors are consistent across both configurations and are therefore attributable to the trained network parameters rather than to the adopted fixed-point format.

When the Tip 1+Tip 2 configuration is enabled, weights and biases are represented using the Q1.7 fixed-point format, reducing fractional precision while preserving sign information. Under this configuration, a higher number of misclassifications is observed during offline functional validation (e.g., digits 0, 3, and 8 misclassified as 5). This behavior reflects the reduced numerical precision available during accumulation and activation stages, which affects the robustness of the classification outcome for borderline inputs.

Runtime classification behavior reflects the actual operating conditions of the UART-based DNN microserver and may reproduce input-dependent anomalies when inconsistent binary samples are used, as discussed in Chapter 8.

When images are streamed via UART, classification results reflect the behavior observed during embedded execution.

Runtime classification reflects the UART-streamed inputs and may reproduce the digit-5 anomaly when the inconsistent binary sample is used; this behavior is addressed and explained in Chapter 8.

Importantly, the observed variations in classification behavior do not affect runtime performance. Experimental measurements confirm that inference execution time remains deterministic and invariant across all fixed-point configurations. The impact of Tip 2 is therefore confined to numerical robustness and classification accuracy under offline validation, without introducing any additional computational or communication overhead.

Overall, these results highlight a clear trade-off between numerical precision and classification robustness, while confirming that the proposed optimizations preserve deterministic timing behavior and real-time capability of the embedded DNN inference system, as summarized in Table 9.2.

Table 9.2: Classification accuracy under different fixed-point configurations. Offline validation is always performed on Q8.8 embedded images, while runtime accuracy refers to UART-streamed inputs (digits 0–9).

Configuration	Evaluation Mode	Correct	Incorrect	Accuracy [%]
Baseline (Q8.8)	Offline (Q8.8 embedded images)	9 / 10	1 / 10	90%
Baseline (Q8.8)	Runtime (UART Q8.8 streaming)	9 / 10	1 / 10	90%
Tip 1 (Q0.8 input, Q8.8 int.)	Offline (Q8.8 embedded images)	9 / 10	1 / 10	90%
Tip 1 (Q0.8 input, Q8.8 int.)	Runtime (UART Q0.8 streaming)	9 / 10	1 / 10	90%
Tip 1 + Tip 2 (Q0.8 + Q1.7)	Offline (Q8.8 embedded images)	7 / 10	3 / 10	70%
Tip 1 + Tip 2 (Q0.8 + Q1.7)	Runtime (UART Q0.8 streaming)	10 / 10	0 / 10	100%

Runtime classification reflects the UART-streamed inputs and may reproduce input-dependent anomalies when inconsistent binary samples are used; this behavior is analyzed and explained in Chapter 8.

The reduced classification accuracy observed during offline validation under Tip 1+Tip 2 is a direct consequence of the lower fractional precision adopted for weights and biases (Q1.7). While input images are always provided in Q8.8 format during offline testing, the quantization of network parameters introduces small numerical errors in the multiply–accumulate operations, which may affect the classification of borderline samples. This behavior represents a controlled precision–robustness trade-off and does not impact runtime accuracy or deterministic execution behavior.

9.6.5 Discussion on Optimized Configuration

The performance evaluation with optimization tips enabled highlights a key system-level insight. While computational performance remains essentially unchanged across all configurations, communication efficiency is significantly improved through compact input data representation, and numerical behavior is affected in a controlled and predictable manner by reduced-precision parameter encoding.

These results confirm that, in embedded DNN microserver applications, effective optimization strategies primarily target data representation and communication mechanisms rather than raw computational acceleration. In particular, Tip 1 directly addresses the dominant system bottleneck by reducing UART bandwidth requirements, while Tip 2 introduces a deliberate precision–robustness trade-off at the numerical level without impacting execution time or deterministic behavior.

The optimized configuration therefore yields a more balanced system design, in which communication latency is reduced and computational headroom is preserved, while fully satisfying real-time requirements and maintaining predictable execution characteristics under runtime operating conditions.

These observations are consistent with the state of the art in fixed-point and quantized neural network inference, where reduced precision does not necessarily degrade classification outcomes for non-boundary inputs, while significantly improving system-level efficiency.

9.7 Overall Discussion

The performance evaluation presented in this chapter highlights a clear separation between computation and communication costs in the proposed DNN microserver. In both the baseline and optimized configurations, the inference pipeline exhibits deterministic execution times on the order of a few milliseconds per image, confirming the suitability of a fixed-point, software-only implementation for real-time operation on an embedded ARM Cortex-A9 processor.

The baseline analysis demonstrates that computation is not the limiting factor, as inference time is orders of magnitude smaller than UART communication latency. This observation is further reinforced by the optimized configuration, in which communication-oriented enhancements significantly reduce input reception time without affecting computational performance.

The comparison between baseline and optimized results confirms that system-level optimizations targeting data representation and communication bandwidth are more effective than purely computational optimizations in this context. At the same time, the adoption of alternative fixed-point formats for network parameters affects numerical robustness and offline classification behavior, while preserving deterministic timing and full functional correctness under runtime operating conditions.

Overall, the obtained results validate the design choices adopted in this work and demonstrate that careful software design, combined with measurement-driven analysis, enables efficient and scalable deployment of Deep Neural Network inference on resource-constrained embedded platforms. To summarize the key findings of the performance evaluation and their implications from a system design perspective, Table 9.3 reports a compact mapping between experimental observations and the corresponding design choices and conclusions.

Table 9.3: Summary of performance results and corresponding design implications

Observed Result	Experimental Evidence	Design Implication
Inference execution time is stable and deterministic	Measured inference time of ≈ 3 ms with negligible variance across iterations	Fixed-point, software-only DNN inference is suitable for real-time embedded execution without OS support
Computation is not the system bottleneck	Inference time is three orders of magnitude smaller than UART reception time	System performance is communication-bound; computational acceleration would provide limited benefits
UART communication dominates end-to-end latency	UART input reception accounts for more than 99% of total iteration time in baseline configuration	Optimization efforts should primarily target data representation and communication bandwidth
Compact input representation reduces latency	Q0.8 input streaming reduces UART reception time by $\approx 31\%$	Communication-oriented optimizations yield significant system-level performance gains
Reduced-precision parameters preserve runtime correctness	Q1.7 weights and biases do not affect inference time and preserve correct runtime classification behavior	Alternative fixed-point formats enable memory and numerical trade-offs without compromising deterministic timing or real-time operation
Optimizations preserve real-time capability	Inference always completes within the inter-arrival time of input samples	System satisfies real-time requirements with significant timing margins

Chapter 10

Conclusions

This work demonstrates that a fully software-based Deep Neural Network can achieve real-time handwritten digit classification on an embedded ARM Cortex-A9 platform when fixed-point arithmetic and careful system-level design principles are applied. The results confirm that, even in the absence of hardware accelerators or dedicated neural processing units, efficient and deterministic DNN inference is feasible on general-purpose embedded processors.

Starting from the analysis of the network architecture using ONNX models and the Netron visualization tool, the inference pipeline was manually mapped onto the embedded processing system. This process required explicit control over data representation, memory layout, and execution flow, emphasizing key aspects that are often abstracted away in high-level AI frameworks but become critical in resource-constrained embedded environments.

The adoption of fixed-point arithmetic enabled the complete elimination of floating-point operations, ensuring deterministic execution behavior and predictable timing characteristics. This design choice proved particularly effective in a standalone execution model, where low-level control and timing repeatability are essential for accurate performance characterization and real-time validation.

The implementation of a UART-based DNN microserver enabled a realistic evaluation of inference under communication constraints. Experimental results clearly showed that, in the considered configuration, the execution time of the DNN inference is orders of magnitude smaller than the latency introduced by UART-based image transmission. As a consequence, the system operates in a communication-bound regime, while the inference pipeline itself fully satisfies real-time execution requirements with large timing margins.

Cycle-accurate measurements obtained using the ARM Global Timer provided detailed insight into the contribution of each processing stage. Per-layer profiling confirmed that the first fully connected layer dominates the computational workload, in line with theoretical expectations for dense neural networks, and validated the effectiveness of the adopted measurement methodology.

Additional system-level optimizations, including compact input data representation for UART streaming and alternative fixed-point formats for network parameters, significantly improved communication efficiency and enabled controlled numerical trade-offs. In particular, reduced-precision parameter encoding affects numerical robustness under offline validation, while preserving deterministic execution and full functional correctness under runtime operating conditions. The successful extension of the implementation to a custom Group 4 network topology demonstrated the scalability, modularity, and reusability of the proposed software framework.

Overall, this laboratory activity illustrates how Deep Neural Network inference can be effectively deployed on a general-purpose embedded processor through careful software design, deterministic execution models, and measurement-driven optimization. The presented results highlight

the importance of system-level analysis when integrating AI workloads into embedded platforms and provide a solid foundation for future investigations on hardware–software co-design and performance-aware AI deployment in resource-constrained systems.

Chapter 11

Appendix – Preprocessing Tools and Optimization Scripts

This appendix collects the Python tools developed to support the optimization, validation, and experimental evaluation of the DNN microserver implemented in Laboratory 5.

The scripts presented in this appendix are not part of the embedded application executed on the Zybo Z7 platform, but were used offline to: (i) preprocess network parameters and input data, (ii) enable communication-oriented and numerical optimizations, and (iii) validate the correctness and consistency of the experimental setup.

All tools are designed to be deterministic, reproducible, and directly traceable to the corresponding optimizations discussed in the main body of the report.

Table 11.1: Offline preprocessing, validation, and verification tools used in Laboratory 5

Script	Tip	Purpose
<code>bin2weights.py</code>	Tip 2	Conversion of DNN weights and biases from Q8.8 to Q1.7 fixed-point format to reduce memory footprint and enable controlled numerical trade-offs.
<code>tip1.q88.to.q08.py</code>	Tip 1	Compression of MNIST input images from Q8.8 to Q0.8 representation, reducing UART bandwidth requirements by approximately 50%.
<code>images_test_bin.py</code>	Validation	Offline consistency check between statically embedded MNIST images and binary images used for UART-based inference, ensuring numerical equivalence and correct preprocessing.
<code>verify_custom_dnn.py</code>	Validation	Bit-accurate Python reference implementation of the custom Group 4 DNN, used as a golden model to verify weights, biases, fixed-point arithmetic, layer ordering, activation behavior, and final classification results against the embedded C implementation.

11.1 Overview of the Optimization Workflow

The optimization workflow adopted in this work is composed of three main stages:

- offline quantization of input images for UART transmission optimization (Tip 1);
- offline conversion of network weights and biases to alternative fixed-point representations (Tip 2);
- functional validation of the baseline and custom DNN architectures using reference MNIST images.

All preprocessing steps are executed on the host PC using Python, while the embedded platform executes only the fixed-point inference pipeline. This separation allows isolating communication and computation effects and ensures deterministic behavior during runtime measurements.

11.2 Input Image Quantization (TIP 1)

In the baseline configuration, MNIST input images are transmitted over UART using a Q8.8 fixed-point representation, requiring two bytes per pixel. While this format simplifies integration with the internal fixed-point inference pipeline, it introduces a significant communication overhead.

Tip 1 addresses this limitation by adopting a compact Q0.8 representation for input images, where each pixel is encoded using a single byte. The conversion is performed offline using a Python script that exploits the numerical characteristics of MNIST data.

Since MNIST pixel values are normalized in the range $[0, 1)$, their Q8.8 fixed-point representation is expected to lie within the interval $[0x0000, 0x00FF]$.

Under this assumption, the most significant byte is always zero and can be safely discarded during transmission. As a result, UART payload is reduced by approximately 50% without loss of numerical information.

11.2.1 Python Script for Q8.8 to Q0.8 Conversion

Listing 11.1 reports the Python script used to convert MNIST test images from Q8.8 to Q0.8 fixed-point format.

The script processes binary image files in a deterministic order, performs defensive clipping to handle potential out-of-range values, and generates compact binary payloads suitable for UART transmission. All conversions are executed offline on the host PC, ensuring that no additional runtime overhead is introduced on the embedded platform.

```

1  #!/usr/bin/env python3
2  # =====
3  #   tip1_q88_to_q08.py
4  #
5  #   Lab 5      DNN on MNIST (UART Microserver      Tip 1)
6  #   Course: Advanced Embedded Systems (AES)
7  #   Board: Zybo Z7      Zynq-7000 (ARM Cortex-A9)
8  #
9  #   Description:
10 #   -----
11 #   Tip 1      UART Bandwidth Optimization
12 #
13 #   This script converts MNIST test images stored in Q8.8 fixed-point
14 #   format into a compact Q0.8 representation.
15 #
16 #   Author: Lello Molinaro
17 #   =====
18
19 import os
20
21 SRC_DIR = "../test/mnist_digits_bin"
22 DST_DIR = "../test/mnist_digits_q08"
23
24 os.makedirs(DST_DIR, exist_ok=True)
25

```

```

26 for fname in sorted(os.listdir(SRC_DIR)):
27     if not fname.endswith(".bin"):
28         continue
29
30     src_path = os.path.join(SRC_DIR, fname)
31     base, _ = os.path.splitext(fname)
32     dst_path = os.path.join(DST_DIR, base + "_q08.bin")
33
34     with open(src_path, "rb") as f:
35         data = f.read()
36
37     if len(data) % 2 != 0:
38         raise ValueError(f"{fname}: invalid file size")
39
40     q08 = bytearray()
41
42     for i in range(0, len(data), 2):
43         lsb = data[i]
44         msb = data[i + 1]
45         value = (msb << 8) | lsb
46
47         if value > 0x00FF:
48             value = 0x00FF
49
50         q08.append(value & 0xFF)
51
52     with open(dst_path, "wb") as f:
53         f.write(q08)
54
55     print(f"{fname}: {len(data)} bytes      {len(q08)} bytes")
56
57 print("Conversion completed successfully.")

```

Listing 11.1: Python script for offline conversion of MNIST images from Q8.8 to Q0.8 format (UART optimization – Tip 1).

11.3 Weight and Bias Conversion (TIP 2)

Tip 2 focuses on improving numerical robustness and memory efficiency by adopting an alternative fixed-point representation for network weights and biases.

While the baseline implementation uses a Q8.8 fixed-point format, all network parameters are converted offline to a Q1.7 representation in the optimized configuration. The conversion is performed using a dedicated Python script that processes the original binary parameter files generated from the ONNX model.

The adopted conversion strategy consists of:

- arithmetic right shift by one bit to convert Q8.8 values into Q1.7;
- explicit saturation to the signed 8-bit range $[-128, 127]$ to prevent overflow and ensure predictable behavior.

All conversions are executed offline on the host PC. The resulting parameters are embedded directly in the application as static C macros, avoiding any runtime overhead and preserving deterministic execution behavior on the ARM Cortex-A9 processor.

11.3.1 Python Script for Weight and Bias Conversion

Listing 11.2 reports the Python script used to convert binary weight and bias files from the baseline Q8.8 fixed-point format to both Q8.8 and Q1.7 representations.

The script reads little-endian signed 16-bit values, generates C-compatible macros for static initialization, and produces a single header file containing both baseline and optimized param-

eters. This approach enables straightforward comparison between configurations and supports reproducibility of the experimental results.

```
1 #!/usr/bin/env python3
2 # =====
3 # bin2weights.py
4 #
5 # Lab 5      DNN on MNIST (Custom Network      Group 4)
6 # Course: Advanced Embedded Systems (AES)
7 # Board: Zybo Z7      Zynq-7000 (ARM Cortex-A9)
8 #
9 # Description:
10 # -----
11 # This script converts binary weight and bias files (little-endian,
12 # signed 16-bit integers in Q8.8 fixed-point format) into C header
13 # macros suitable for direct inclusion in the embedded DNN
14 # inference code running on the ARM Processing System.
15 #
16 # Two representations are generated:
17 #     Baseline Q8.8
18 #     Tip 2 Q1.7
19 #
20 # Author: Lello Molinaro
21 # =====
22
23 import struct
24
25 def bin_to_define(binfile, name, outfile, comment=None):
26     with open(binfile, "rb") as f:
27         data = f.read()
28
29         values = struct.unpack("<" + "h" * (len(data) // 2), data)
30
31         if comment:
32             outfile.write(f"// {comment}\n")
33
34         outfile.write(f"#define {name} ")
35
36         for i, v in enumerate(values):
37             outfile.write(str(v))
38             if i != len(values) - 1:
39                 outfile.write(",")
40
41         outfile.write("\n\n")
42
43 def bin_to_define_q17(binfile, name, outfile, comment=None):
44     with open(binfile, "rb") as f:
45         data = f.read()
46
47         values_q88 = struct.unpack("<" + "h" * (len(data) // 2), data)
48
49         values_q17 = []
50         for v in values_q88:
51             x = v >> 1
52             x = max(-128, min(127, x))
53             values_q17.append(x)
54
55         if comment:
56             outfile.write(f"// {comment} (Q1.7 Tip 2)\n")
57
58         outfile.write(f"#define {name}_q17 ")
59
60         for i, v in enumerate(values_q17):
61             outfile.write(str(v))
62             if i != len(values_q17) - 1:
63                 outfile.write(",")
64
65         outfile.write("\n\n")
```

Listing 11.2: Python script for offline conversion of DNN weights and biases from Q8.8 to Q1.7 (Tip 2).

11.4 Baseline vs Custom Network Validation

Functional validation was performed using a fixed set of reference MNIST images embedded directly in the application as static C macros. This validation step was used to verify the correctness of the inference pipeline independently of UART communication and fixed-point optimization settings.

In addition to runtime validation on the embedded platform, an offline consistency check was performed to verify the numerical equivalence between: (i) MNIST images embedded in the C source code, and (ii) MNIST images stored as raw binary files used for UART-based inference.

This additional verification step ensures that no unintended scaling, truncation, or endianness issues were introduced during image preprocessing, format conversion, or UART transmission.

11.4.1 Offline Image Consistency Verification

To validate the consistency between embedded test images and binary images used for UART-based inference, a dedicated Python script was developed.

The script parses MNIST test images defined as C macros in `test_images.h`, loads the corresponding binary image files, automatically detects the fixed-point scale (Q8.8 or Q0.8), and computes pixel-wise differences.

Both numerical and visual comparisons are performed. For each digit, the script reports the maximum absolute difference and the number of non-zero differences, and generates diagnostic images showing the embedded image, the binary image, and the absolute difference.

This procedure guarantees that all images used for functional and performance testing are numerically consistent across representations.

```
1  #!/usr/bin/env python3
2  # =====
3  #  images_test_bin.py
4  #
5  #  Lab 5      DNN on MNIST (Verification & Debug Utility)
6  #  Course: Advanced Embedded Systems (AES)
7  #  Board: Zybo Z7      Zynq-7000 (ARM Cortex-A9)
8  #
9  #  Description:
10 #  -----
11 #  This script performs a consistency check between:
12 #    1) MNIST test images embedded in C code via macros
13 #    2) MNIST images stored as raw binary files (.bin)
14 #
15 #  Author: Lello Molinaro
16 #  =====
17
18 import re
19 import numpy as np
20 from pathlib import Path
21 import matplotlib.pyplot as plt
22
23 TEST_IMAGES_H = r"path/to/test_images.h"
24 BIN_DIR = r"path/to/mnist_digits_bin"
25
26 OUT_IMG_DIR = Path("../comparison_images")
27 OUT_IMG_DIR.mkdir(exist_ok=True)
28
29 def parse_test_images_h(path):
30     with open(path, "r", encoding="utf-8") as f:
```

```

31         content = f.read()
32
33     images = {}
34     pattern = re.compile(r"#define\s+imm_test_(\d+)\s+([\n\r]+)")
35     matches = pattern.findall(content)
36
37     for idx, data in matches:
38         tokens = [t.strip() for t in data.split(",") if t.strip()]
39         values = [int(v) for v in tokens]
40         images[int(idx)] = np.array(values, dtype=np.int16)
41
42     return images
43
44 def load_bin_image(path):
45     img = np.fromfile(path, dtype=np.int16)
46     return img
47
48 c_images = parse_test_images_h(TEST_IMAGES_H)
49
50 for digit in range(10):
51     bin_path = Path(BIN_DIR) / f"{digit}.bin"
52     c_img = c_images[digit]
53     bin_img = load_bin_image(bin_path)
54
55     scale = 256 if np.max(np.abs(bin_img)) > 255 else 1
56     bin_img_scaled = bin_img // scale
57     diff = bin_img_scaled - c_img
58
59     print(f"Digit {digit}: max diff = {np.max(np.abs(diff))}")
60
61     fig, axs = plt.subplots(1, 3, figsize=(9, 3))
62     axs[0].imshow(c_img.reshape(28, 28), cmap="gray")
63     axs[1].imshow(bin_img_scaled.reshape(28, 28), cmap="gray")
64     axs[2].imshow(np.abs(diff).reshape(28, 28), cmap="hot")
65     for ax in axs:
66         ax.axis("off")
67
68     plt.tight_layout()
69     plt.savefig(OUT_IMG_DIR / f"digit_{digit}_compare.png", dpi=150)
70     plt.close()

```

Listing 11.3: Python script for offline consistency check between embedded MNIST images and binary images used for UART-based inference.

11.4.2 Python DNN Golden Model

In order to validate the correctness of the embedded fixed-point inference pipeline, a bit-accurate Python reference implementation of the DNN was developed and used as a golden model.

The script reproduces exactly the computation performed on the ARM Cortex-A9, including: (i) fixed-point arithmetic in Q8.8 format, (ii) explicit multiply-accumulate loops, (iii) bias handling and scaling via arithmetic shifts, and (iv) ReLU activation and final classification.

The Python model loads the same binary weights, biases, and input images used by the embedded application and performs inference layer by layer using int16 arithmetic. This enables direct comparison between Python and C results and allows isolating numerical or implementation errors independently of the hardware platform.

```

1  #!/usr/bin/env python3
2  # =====
3  #   dnn_python_reference.py
4  #
5  #   Lab 5      DNN on MNIST (Python Golden Model)
6  #   Course: Advanced Embedded Systems (AES)
7  #   Board: Zybo Z7      Zynq-7000 (ARM Cortex-A9)
8  #
9  #   Description:
10 #   -----

```

```

11 # This script implements a **bit-accurate Python reference model**
12 # of the DNN used in Lab 5, with the explicit goal of validating
13 # the correctness of:
14 #
15 #     - weights and biases exported from training
16 #     - fixed-point arithmetic (Q8.8)
17 #     - layer ordering and dimensions
18 #     - ReLU activation behavior
19 #     - final classification results
20 #
21 # The implementation mirrors *exactly* the C code running on the
22 # ARM Cortex-A9:
23 #     - int16 arithmetic
24 #     - explicit MAC loops
25 #     - fixed-point scaling via shifts
26 #
27 # This script therefore acts as a **golden model** for debugging
28 # and cross-checking the embedded implementation.
29 #
30 # Author: Lello Molinaro
31 # Academic Year: 2025 2026
32 # =====
33
34 import numpy as np
35 import os
36
37
38 # =====
39 # PARAMETERS
40 # =====
41 QF = 8 # Fractional bits for Q8.8 fixed-point format
42 IMG_SIZE = 784 # MNIST image size (28 28)
43 BASE = "mnist_digits_bin" # Directory containing test images
44
45
46 # =====
47 # LOAD BINARIES (INT16, LITTLE-ENDIAN)
48 # =====
49 def load_bin(path):
50     """
51     Load a raw binary file containing int16 values
52     encoded in little-endian format.
53     """
54     return np.fromfile(path, dtype=np.int16)
55
56
57 # -----
58 # Bias vectors
59 # -----
60 b0 = load_bin("../group_4/weights/Gemm0_biases.bin")
61 b1 = load_bin("../group_4/weights/Gemm1_biases.bin")
62 b2 = load_bin("../group_4/weights/Gemm2_biases.bin")
63 b3 = load_bin("../group_4/weights/Gemm3_biases.bin")
64
65 # -----
66 # Weight matrices (row-major layout)
67 # -----
68 # Dimensions follow the custom Group 4 topology:
69 #   FC0: 784 64
70 #   FC1: 64 32
71 #   FC2: 32 16
72 #   FC3: 16 10
73 w0 = load_bin("../group_4/weights/Gemm0_weights.bin").reshape(64, 784)
74 w1 = load_bin("../group_4/weights/Gemm1_weights.bin").reshape(32, 64)
75 w2 = load_bin("../group_4/weights/Gemm2_weights.bin").reshape(16, 32)
76 w3 = load_bin("../group_4/weights/Gemm3_weights.bin").reshape(10, 16)
77
78
79 # =====
80 # LOAD TEST IMAGES

```

```

81 # =====
82 images = []
83 expected = []
84
85 for d in range(10):
86     img = load_bin(os.path.join(BASE, f"{d}.bin"))
87
88     # Each MNIST image must contain exactly 784 samples
89     assert img.size == IMG_SIZE, f"{d}.bin size error"
90
91     images.append(img)
92     expected.append(d)
93
94
95 # =====
96 # DNN OPERATIONS (BIT-ACCURATE WITH C CODE)
97 # =====
98 def fc_forward(x, W, b):
99     """
100     Fully-connected layer forward pass.
101
102     This function mirrors the C implementation:
103     - Accumulator initialized as (bias << QF)
104     - MAC in int32 precision
105     - Final right shift by QF
106     - Output cast to int16
107
108     Args:
109         x (np.ndarray): input vector (int16)
110         W (np.ndarray): weight matrix (int16), shape (out, in)
111         b (np.ndarray): bias vector (int16)
112
113     Returns:
114         np.ndarray: output vector (int16)
115     """
116     out = np.zeros(W.shape[0], dtype=np.int16)
117
118     for o in range(W.shape[0]):
119         mac = int(b[o]) << QF
120         for i in range(W.shape[1]):
121             mac += int(x[i]) * int(W[o, i])
122         out[o] = np.int16(mac >> QF)
123
124     return out
125
126
127 def relu(x):
128     """
129     ReLU activation function.
130     Matches the embedded implementation exactly.
131     """
132     return np.maximum(x, 0).astype(np.int16)
133
134
135 def argmax(x):
136     """
137     Return index of maximum value (class prediction).
138     """
139     return int(np.argmax(x))
140
141
142 # =====
143 # TEST / VERIFICATION
144 # =====
145 print("\n==== PYTHON DNN VERIFICATION =====\n")
146
147 for i in range(10):
148     x = images[i]
149
150     # FCO + ReLU

```

```

151     x = fc_forward(x, w0, b0)
152     x = relu(x)
153
154     # FC1 + ReLU
155     x = fc_forward(x, w1, b1)
156     x = relu(x)
157
158     # FC2 + ReLU
159     x = fc_forward(x, w2, b2)
160     x = relu(x)
161
162     # FC3 (logits)
163     logits = fc_forward(x, w3, b3)
164
165     pred = argmax(logits)
166
167     print(f"Digit {i}: expected={i} predicted={pred}")
168     print("logits:", logits.tolist())
169     print()

```

Listing 11.4: Bit-accurate Python reference implementation of the DNN used as a golden model for validation.

11.5 Runtime Output Log of Experimental Results

This section reports an excerpt of the runtime output log (`results.txt`) generated during the experimental evaluation of the UART-based DNN microserver. The log was captured directly from the serial console during execution on the Zybo Z7 platform and reflects the real-time behavior of the embedded application under different fixed-point configurations.

The reported output includes: (i) the active fixed-point configuration, (ii) functional validation on statically linked MNIST test images, and (iii) cycle-accurate timing measurements for UART reception, DNN inference, and total iteration time. The log therefore provides a complete and transparent trace of both functional correctness and performance measurements discussed in Chapters 6 and 9.

The excerpt below shows representative outputs for:

- the baseline configuration (Q8.8 input and parameters),
- the UART-optimized configuration (Tip 1, Q0.8 input),
- the combined optimized configuration (Tip 1 + Tip 2, Q0.8 input and Q1.7 parameters).

```

1
2
3
4 Started
5
6
7 === LAB5 === DNN on MNIST===
8
9
10 === GROUP 4 CUSTOM DNN on MNIST===
11
12
13 -----
14 Fixed-Point Configuration
15 -----
16 Mode          : BASELINE
17 Input         : Q8.8   (2 bytes / pixel)
18 Weights       : Q8.8
19 Bias          : Q8.8
20 Internal QF   : 8
21 -----
22
23

```

```
24 ===== Test image 0 =====
25 Expected = 0, Predicted = 0
26
27 ===== Test image 1 =====
28 Expected = 1, Predicted = 1
29
30 ===== Test image 2 =====
31 Expected = 2, Predicted = 2
32
33 ===== Test image 3 =====
34 Expected = 3, Predicted = 3
35
36 ===== Test image 4 =====
37 Expected = 4, Predicted = 4
38
39 ===== Test image 5 =====
40 Expected = 5, Predicted = 3
41
42 ===== Test image 6 =====
43 Expected = 6, Predicted = 6
44
45 ===== Test image 7 =====
46 Expected = 7, Predicted = 7
47
48 ===== Test image 8 =====
49 Expected = 8, Predicted = 8
50
51 ===== Test image 9 =====
52 Expected = 9, Predicted = 9
53
54 Waiting for image...
55 Predicted digit = 0
56
57 TIMING us: RX=9095894 DNN=2989 PRINT=18 ITER=9098902
58
59
60 Waiting for image...
61 Predicted digit = 1
62
63 TIMING us: RX=3568231 DNN=2988 PRINT=18 ITER=3571239
64
65
66 Waiting for image...
67 Predicted digit = 2
68
69 TIMING us: RX=3105781 DNN=2988 PRINT=17 ITER=3108787
70
71
72 Waiting for image...
73 Predicted digit = 3
74
75 TIMING us: RX=3226787 DNN=2988 PRINT=17 ITER=3229794
76
77
78 Waiting for image...
79 Predicted digit = 4
80
81 TIMING us: RX=3213994 DNN=2989 PRINT=18 ITER=3217002
82
83
84 Waiting for image...
85 Predicted digit = 3
86
87 TIMING us: RX=3711176 DNN=2988 PRINT=17 ITER=3714182
88
89
90 Waiting for image...
91 Predicted digit = 6
92
93 TIMING us: RX=3585874 DNN=2988 PRINT=18 ITER=3588881
```

```

94
95
96 Waiting for image...
97 Predicted digit = 7
98
99 TIMING us: RX=3167111 DNN=2987 PRINT=18 ITER=3170117
100
101
102 Waiting for image...
103 Predicted digit = 8
104
105 TIMING us: RX=3491976 DNN=2989 PRINT=18 ITER=3494984
106
107
108 Waiting for image...
109 Predicted digit = 9
110
111 TIMING us: RX=3181613 DNN=2988 PRINT=19 ITER=3184620
112
113
114 Waiting for image...
115
116
117 Started
118
119
120 === LAB5 === DNN on MNIST===
121
122
123 === GROUP 4 CUSTOM DNN on MNIST===
124
125
126 -----
127 Fixed-Point Configuration
128 -----
129 Mode      : TIP 1 (UART optimization)
130 Input     : Q0.8 (1 byte / pixel)
131 Weights   : Q8.8
132 Bias      : Q8.8
133 Internal QF : 8
134 -----
135
136
137 ===== Test image 0 =====
138 Expected = 0, Predicted = 0
139
140 ===== Test image 1 =====
141 Expected = 1, Predicted = 1
142
143 ===== Test image 2 =====
144 Expected = 2, Predicted = 2
145
146 ===== Test image 3 =====
147 Expected = 3, Predicted = 3
148
149 ===== Test image 4 =====
150 Expected = 4, Predicted = 4
151
152 ===== Test image 5 =====
153 Expected = 5, Predicted = 3
154
155 ===== Test image 6 =====
156 Expected = 6, Predicted = 6
157
158 ===== Test image 7 =====
159 Expected = 7, Predicted = 7
160
161 ===== Test image 8 =====
162 Expected = 8, Predicted = 8
163

```

```
164 |===== Test image 9 =====
165 |Expected = 9, Predicted = 9
166 |
167 |Waiting for image...
168 |Predicted digit = 0
169 |
170 |TIMING us: RX=9268721 DNN=2989 PRINT=18 ITER=9271728
171 |
172 |
173 |Waiting for image...
174 |Predicted digit = 1
175 |
176 |TIMING us: RX=3024197 DNN=2988 PRINT=18 ITER=3027205
177 |
178 |
179 |Waiting for image...
180 |Predicted digit = 2
181 |
182 |TIMING us: RX=2586165 DNN=2988 PRINT=17 ITER=2589171
183 |
184 |
185 |Waiting for image...
186 |Predicted digit = 3
187 |
188 |TIMING us: RX=2963065 DNN=2988 PRINT=17 ITER=2966072
189 |
190 |
191 |Waiting for image...
192 |Predicted digit = 4
193 |
194 |TIMING us: RX=2570522 DNN=2988 PRINT=18 ITER=2573529
195 |
196 |
197 |Waiting for image...
198 |Predicted digit = 3
199 |
200 |TIMING us: RX=2697534 DNN=2988 PRINT=18 ITER=2700541
201 |
202 |
203 |Waiting for image...
204 |Predicted digit = 6
205 |
206 |TIMING us: RX=2759715 DNN=2988 PRINT=18 ITER=2762722
207 |
208 |
209 |Waiting for image...
210 |Predicted digit = 7
211 |
212 |TIMING us: RX=2775086 DNN=2989 PRINT=18 ITER=2778094
213 |
214 |
215 |Waiting for image...
216 |Predicted digit = 8
217 |
218 |TIMING us: RX=2538270 DNN=2988 PRINT=18 ITER=2541277
219 |
220 |
221 |Waiting for image...
222 |Predicted digit = 9
223 |
224 |TIMING us: RX=2697155 DNN=2988 PRINT=19 ITER=2700163
225 |
226 |
227 |Waiting for image...
228 |
229 |
230 |Started
231 |
232 |
233 |=== LAB5 === DNN on MNIST===
```

```

234
235
236 === GROUP 4 CUSTOM DNN on MNIST===
237
238
239 -----
240 Fixed-Point Configuration
241 -----
242 Mode          : TIP 1 + TIP 2
243 Input         : Q0.8  (1 byte / pixel)
244 Weights       : Q1.7
245 Bias          : Q1.7
246 Internal QF   : 7
247 -----
248
249
250 ===== Test image 0 =====
251 Expected = 0, Predicted = 5
252
253 ===== Test image 1 =====
254 Expected = 1, Predicted = 1
255
256 ===== Test image 2 =====
257 Expected = 2, Predicted = 2
258
259 ===== Test image 3 =====
260 Expected = 3, Predicted = 5
261
262 ===== Test image 4 =====
263 Expected = 4, Predicted = 4
264
265 ===== Test image 5 =====
266 Expected = 5, Predicted = 5
267
268 ===== Test image 6 =====
269 Expected = 6, Predicted = 6
270
271 ===== Test image 7 =====
272 Expected = 7, Predicted = 7
273
274 ===== Test image 8 =====
275 Expected = 8, Predicted = 5
276
277 ===== Test image 9 =====
278 Expected = 9, Predicted = 9
279
280 Waiting for image...
281 Predicted digit = 0
282
283 TIMING us: RX=2300112 DNN=2989 PRINT=18 ITER=2303119
284
285
286 Waiting for image...
287 Predicted digit = 1
288
289 TIMING us: RX=3024872 DNN=2987 PRINT=18 ITER=3027878
290
291
292 Waiting for image...
293 Predicted digit = 2
294
295 TIMING us: RX=2932178 DNN=2988 PRINT=17 ITER=2935185
296
297
298 Waiting for image...
299 Predicted digit = 3
300
301 TIMING us: RX=2712982 DNN=2989 PRINT=17 ITER=2715989
302
303

```

```
304 | Waiting for image...
305 | Predicted digit = 4
306 |
307 | TIMING us: RX=2915089 DNN=2988 PRINT=18 ITER=2918096
308 |
309 |
310 | Waiting for image...
311 | Predicted digit = 5
312 |
313 | TIMING us: RX=2977056 DNN=2988 PRINT=17 ITER=2980062
314 |
315 |
316 | Waiting for image...
317 | Predicted digit = 6
318 |
319 | TIMING us: RX=2682027 DNN=2987 PRINT=18 ITER=2685033
320 |
321 |
322 | Waiting for image...
323 | Predicted digit = 7
324 |
325 | TIMING us: RX=2839565 DNN=2988 PRINT=18 ITER=2842572
326 |
327 |
328 | Waiting for image...
329 | Predicted digit = 8
330 |
331 | TIMING us: RX=2851997 DNN=2988 PRINT=18 ITER=2855005
332 |
333 |
334 | Waiting for image...
335 | Predicted digit = 9
336 |
337 | TIMING us: RX=2569450 DNN=2987 PRINT=19 ITER=2572456
338 |
339 |
340 | Waiting for image...
```

Listing 11.5: Excerpt from `results.txt`: runtime serial output for baseline, Tip 1, and Tip 1+Tip 2 configurations.

The complete `results.txt` file (available at Github repository)¹, contains repeated iterations for all test images and configurations and was used to compute the average timing values reported in Chapter 9. The log also supports the validation and debugging analysis discussed in Chapter 8 by providing direct evidence of systematic misclassification and its resolution under controlled input conditions.

¹<https://github.com/lmolinario/AES>